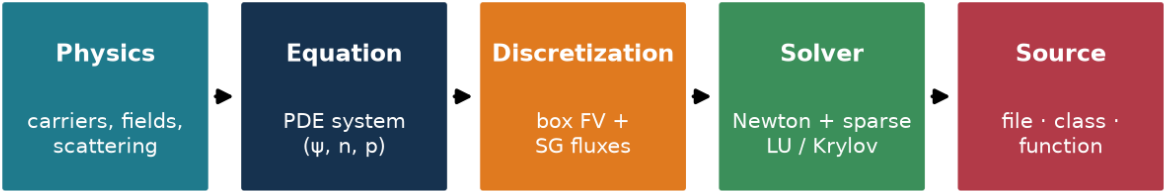


Semiconductor Physics Behind PyTCAD

A first-principles guide that follows every idea from physics → equation → discretization → solver → source code



Every section of this guide ends with a table that walks this chain for the topic just covered.

Subject	Repository nihaltodurkar-cms/NT-SemTCAD — project root pytcad/ (numerical core, C++ engine, workbench, backend service, desktop app)
Basis	Source read on the current branch (Oct 2026). Every file / class / function named in this guide was opened and confirmed. Figures 2-6 and 8 were produced by running the repository's own code; Figures 1, 7 and 9-11 are schematic diagrams.
Platform	Windows only (project rule, CLAUDE.md). Figures in this guide were generated in a Linux session container as a local pre-check; they illustrate physics, not Windows CI results.
Status tags	[IMPLEMENTED] [PARTIALLY IMPLEMENTED] [NOT IMPLEMENTED] — used on every feature, judged from the code, the tests and ARCHITECTURE.md.

A learning document, not a validation report: it explains what the code does and why. Where a number is quoted it is either computed by the repository code (and says so) or cited from a source comment in the code.

Contents

0. How to use this guide	4
1. Semiconductor basics	5
1.1 Bands, bandgap and the picture you must hold in your head	5
1.2 Density of states and carrier statistics	5
1.3 Doping, ionization and bandgap narrowing	6
2. Poisson's equation and electrostatics	6
2.1 What the equation predicts: the p-n junction	6
2.2 Equilibrium solve in the code	7
2.3 Dielectrics and the MOS gate	7
3. Drift-diffusion transport and the continuity equations	8
3.1 Where current comes from	8
3.2 Continuity (conservation of carriers)	8
3.3 Static device physics: the ideal diode	8
4. Generation, recombination and mobility	9
4.1 Recombination	9
4.2 Mobility	10
5. Boundary conditions and contacts	11
5.1 Ohmic contact (Dirichlet)	11
5.2 Insulating boundaries and symmetry planes	11
5.3 Gate (Robin) and Schottky contacts	11
5.4 How a Dirichlet row is imposed numerically	12
6. Devices: the physics that actually appears in the repo	13
6.1 P-N junction and diode	13
6.2 MOS capacitor	13
6.3 MOSFET and multi-gate devices (2D and 3D)	13
6.4 Heterojunctions, HBT, HEMT	14
6.5 Schottky diode	14
6.6 Power and other structures present in the repo	14
6.7 Mixed-mode circuits	14
7. From equations to a discrete system	15
7.1 Scaling (non-dimensionalization)	15
7.2 Meshing	15
7.3 Finite volume (box integration) — not finite element	15
7.4 Scharfetter–Gummel: the key idea	16
7.5 The Jacobian	17
7.6 Newton–Raphson with damping	18
7.7 Sparse direct and iterative solvers; PETSc	18
8. Beyond a single bias point	20

8.1 Bias continuation and arc-length continuation	20
8.2 Transient simulation	20
8.3 Small-signal AC analysis	20
8.4 Self-heating (electro-thermal)	20
9. Solver architecture: 1D, 2D and 3D	21
9.1 A single bias solve, end to end (1D)	21
9.2 What lives outside the numerical core	21
10. Advanced physics models	22
10.1 Impact ionization and avalanche breakdown	22
10.2 Band-to-band tunneling (BTBT)	22
10.3 Tunneling through oxides and barriers	23
10.4 Quantum confinement: density gradient and Schrödinger-Poisson	23
10.5 Hot carriers: energy balance and the hydrodynamic closure	23
11. Process simulation: where the doping comes from	24
12. Verification: how you know the physics is right	25
13. Physics implemented in the current TCAD	26
14. Missing and future physics	28
15. Physics learning dependency map	29
Suggested study path (with repository exercises)	29
16. Semiconductor / TCAD equation cheat sheet	30
17. Repository architecture map	31
Physics → file lookup	31

0. How to use this guide

PyTCAD is a **self-consistent drift-diffusion** device simulator (1D, 2D, 3D) with process simulation, a compiled C++ engine (`pytcad._core`) and a Python/Qt desktop front end. Its governing system is the **van Roosbroeck** set: one Poisson equation plus two carrier continuity equations, solved fully coupled by Newton's method on a box-integration (finite-volume) discretization with **Scharfetter-Gummel** currents.

The guide is ordered from fundamentals to advanced: semiconductor basics (§1) → electrostatics (§2) → transport (§3) → generation/recombination and mobility (§4) → contacts (§5) → devices (§6) → numerical methods (§7–§8) → solver architecture (§9) → advanced physics (§10) → process simulation (§11) → verification (§12). The five closing chapters (§13–§17) are the summaries you asked for.

Reading conventions

- **[IMPLEMENTED]** the feature exists, is exercised by tests, and works in the stated dimensions.
- **[PARTIALLY IMPLEMENTED]** it exists with a documented restriction (a dimension, a model combination, a missing calibration, or an open gate).
- **[NOT IMPLEMENTED]** not in the code (or explicitly out of scope per `ARCHITECTURE.md` §7).
- **Code-map tables** end each topic. Columns: Physics · Equation · Discretization · Solver · Source. File paths are relative to `pytcad/`; C++ lives under `core/`; the Python numerical core under `pytcad/pytcad/`.
- **Unit convention of the library** (from `pytcad/constants.py`): length cm · concentration cm^{-3} · potential V · current density A/cm^2 · time s. Internally everything is *scaled* (§7.1).

NOTE Honest scope

PyTCAD is explicitly a *learning and research* TCAD with published-value gates (see `ARCHITECTURE.md`). It is not foundry-calibrated. The guide therefore tells you what each model neglects (the “Limits” statements are taken from the code's own docstrings).

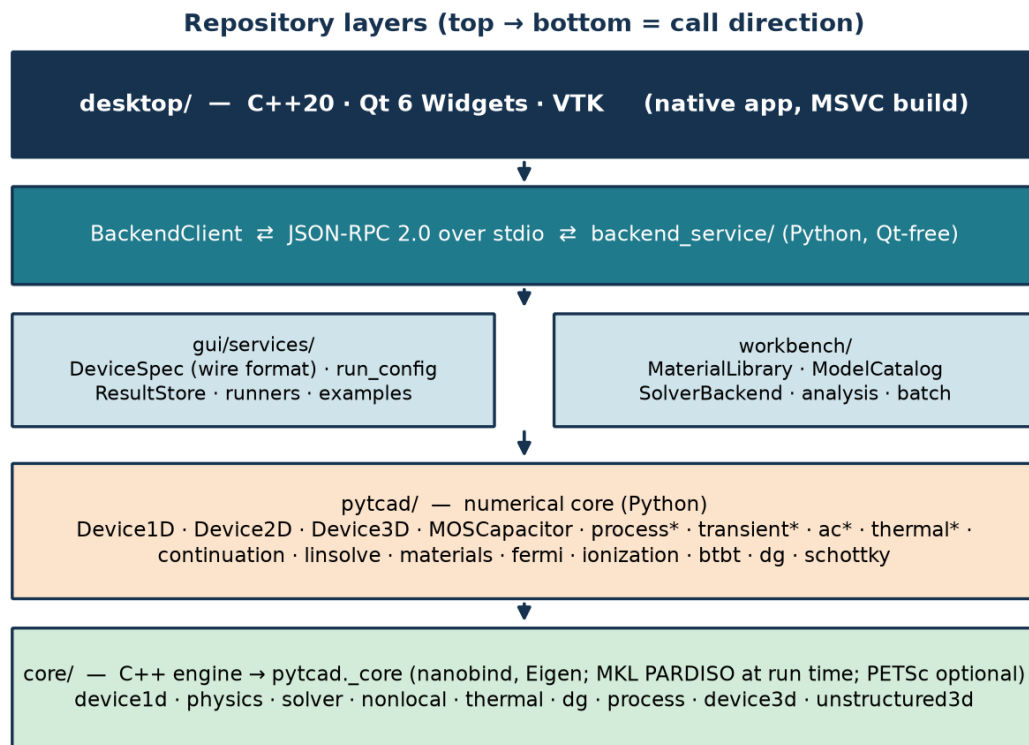


Figure 1 — Repository layers. The physics lives in `pytcad/` and `core/`; everything above is delivery. See §17 for the file-level map.

1. Semiconductor basics

1.1 Bands, bandgap and the picture you must hold in your head

A crystal's periodic potential splits the electron energy spectrum into **bands**. At 0 K the valence band (E_v and below) is full and the conduction band (E_c and above) is empty. The **bandgap** $E_g = E_c - E_v$ separates them. Electrons promoted across the gap leave behind **holes**: empty valence states that behave as positive carriers with their own effective mass. TCAD never tracks individual electrons; it tracks the *densities* $n(x)$ and $p(x)$ of conduction-band electrons and valence-band holes, plus the electrostatic potential $\psi(x)$ that bends the bands.

$$E_g(T) = E_g(0) - \frac{\alpha T^2}{T + \beta} \quad (1.1)$$

Varshni law — implemented as `Semiconductor.Eg(T)` in `pytcad/materials.py`. Silicon defaults (same file): $E_g(0)=1.17$ eV, $\alpha=4.73 \times 10^{-4}$ eV/K, $\beta=636$ K, giving **$E_g(300\text{ K}) = 1.1245$ eV** (computed by the repository code). **[IMPLEMENTED]**

PHYSICAL INTUITION Bands bend, the Fermi level does not

In equilibrium the Fermi level E_F is flat across the whole device. Where the electrostatic potential ψ rises, *all* band edges move down by $q\psi$ (an electron's energy is $-q\psi + E_{c0}$). Everything in a pn junction — depletion, built-in voltage, rectification — follows from a flat E_F and bending bands. The solver's primary unknown ψ is this band bending.

1.2 Density of states and carrier statistics

Near a band edge, the density of states is parabolic: $g_c(E) \propto m^*{}^{3/2} \sqrt{E - E_c}$. Integrating against the Fermi-Dirac occupation $f(E)=1/(1+e^{(E-E_F)/kT})$ gives the carrier densities. Defining the reduced Fermi level $\eta_n = (E_{F,n} - E_c)/kT$ and $\eta_p = (E_v - E_{F,p})/kT$:

$$n = N_c \mathcal{F}_{1/2}(\eta_n), \quad p = N_v \mathcal{F}_{1/2}(\eta_p), \quad N_{c,v} = 2 \left(\frac{2\pi m_{n,p}^* kT}{h^2} \right)^{3/2} \quad (1.2)$$

where $\mathcal{F}_{1/2}$ is the (normalized) Fermi-Dirac integral. In the **non-degenerate (Boltzmann) limit**, $\eta \ll -1$, $\mathcal{F}_{1/2}(\eta) \rightarrow e^\eta$:

$$n = N_c e^{(E_F - E_c)/kT} = n_i e^{(E_F - E_i)/kT}, \quad p = n_i e^{(E_i - E_F)/kT}, \quad np = n_i^2 \quad (1.3)$$

In the repository: **Boltzmann is the default**; Fermi-Dirac is switched on with `Models(fd=True)`. The DOS is not integrated from first principles: `Semiconductor.Nc(T)`, `Nv(T)` use $N_{c,v}(T) = N_{c,v}(300)(T/300)^{3/2}$ with Si values $N_c=2.86 \times 10^{19}$, $N_v=3.10 \times 10^{19}$ cm⁻³ (the effective masses $m_n^*=0.26$, $m_p^*=0.386$ are stored for the quantum/tunneling models). The intrinsic density is

$$n_i = \sqrt{N_c N_v} \exp\left(-\frac{E_g}{2kT}\right) \quad (1.4)$$

`(Semiconductor.ni(T)) = 1.067 × 1010 cm-3` for Si at 300 K (computed by the repo; thermal voltage $V_T=kT/q = 25.852$ mV from `constants.thermal_voltage`).

Fermi-Dirac implementation [IMPLEMENTED]: `pytcad/fermi.py` provides `f_half`, `f_mhalf`, `df_half` and the inverse `f_half_inv` through a tabulated, Hermite-interpolated evaluation (exact reference `f_half_exact` for gating); the C++ mirror is `core/include/tcad/physics/fermi.hpp` (`fd_density`, `fd_node_factors_node`). Documented hazard: the code warns when doping $> 10^{19}$ cm⁻³ is used with Boltzmann statistics (`Device1D.__init__`), because Boltzmann then overestimates carrier density.

PHYSICAL INTUITION Why $np = n_i^2$ matters

The mass-action law is detailed balance: at equilibrium thermal generation across the gap equals recombination. Everything departing from it ($np \neq n_i^2$) is excess carriers, which drives recombination (§4) and current. The SRH and Auger terms in the code are literally proportional to $(np - n_i^2)$ and vanish at equilibrium.

1.3 Doping, ionization and bandgap narrowing

Donors (n-type) and acceptors (p-type) add or remove mobile carriers. With full ionization the charge-neutral bulk satisfies $n - p = C$ where $C = N_D - N_A$ is the *net* doping (library convention: **positive** = **n-type**; e.g. `np.where(x < 1e-4, -1e17, 1e17)` is a p-n step). Solving $n - p = C$ with $np = n_{ie}^2$ gives the majority density used at every ohmic contact (§5):

$$n_0 = \frac{1}{2} \left(C + \sqrt{C^2 + 4n_{ie}^2} \right), \quad p_0 = \frac{n_{ie}^2}{n_0} \quad (C \geq 0) \quad (1.5)$$

Incomplete ionization [IMPLEMENTED] (`Models(incomplete_ion=True)`; M13, lifted to 2D/3D in M41): dopants sit ≈ 45 meV from the band edge, so at 300 K not all are ionized. The ionized fraction uses degeneracy factors $g_D=2$ and $g_A=4$: $N_D^+ = N_D / (1 + g_D e^{(E_F - E_D)/kT})$. Code: `ionized_doping`, `ionized_eta_doping`, `ionized_dE_kt` in `pytcad/device.py`; C++ `tcad::physics::ionized_doping`. *Limits* (code docstring): hydrogenic model, invalid above the Mott transition ($\sim 4 \times 10^{18} \text{ cm}^{-3}$) and for compensated profiles (a net-doping input carries no species split).

Bandgap narrowing [IMPLEMENTED] (Slotboom/de Graaff, `Models(bgn=True)`, default **on**): at heavy doping the gap shrinks, enlarging the effective intrinsic density n_{ie} . The code implements

$$\Delta E_g = E_0 \left[\ln \frac{N}{N_0} + \sqrt{\ln^2 \frac{N}{N_0} + \frac{1}{2}} - \sqrt{\frac{1}{2}} \right] \quad (N > N_0), \quad n_{ie}^2 = n_i^2 e^{\Delta E_g/kT} \quad (1.6)$$

$E_0 = 6.92$ meV, $N_0 = 1.3 \times 10^{17} \text{ cm}^{-3}$ (`materials.bandgap_narrowing_slotboom`, `nie_effective`). The $-\sqrt{1/2}$ shift is the repo's own deviation from the textbook form, added so the clamp at N_0 introduces no jump in n_{ie} (a jump would act as a fictitious field). Effect: more minority-carrier injection from heavy emitters — the dominant reason real BJT gain falls below the ideal-diode value.

Physics → equation → discretization → solver → code: bands, statistics, doping

Physics	Equation	Discretization	Solver	Source (file · class/function)
Bandgap vs T	$E_g(T)$ Varshni	pointwise	n/a	materials.py: Semiconductor.Eg
DOS, n_i	$N_{c,v} \propto T^{3/2}$; $n_i = \sqrt{N_c N_v} e^{-E_g/2kT}$	per-node arrays (scaled by N_s)	n/a	Semiconductor.Nc/Nv/ni ; core/src/device1d/inputs.cpp (<code>nc_s</code> , <code>nv_s</code>)
Fermi-Dirac	$n = N_c F_{1/2}(\eta)$	ν -factor generalized SG (§7.4)	Newton with extra Jacobian columns w_n, w_p	fermi.py; physics/fermi.hpp; device1d.cpp residual_jacobian (if (fd))
Incomplete ionization	$N_D^+(n,p)$	net ionized doping per node	Poisson row gains $\partial C_{ion}/\partial n, \partial p$	device.py ionized_doping ; device2d/3d _ionized_C
BGN	Slotboom $\Delta E_g(N)$	n_{ie} per node	n/a	materials.py bandgap_narrowing_slotboom

2. Poisson's equation and electrostatics

Charge creates field and field bends bands. Gauss's law in a medium of permittivity ϵ (here $\epsilon = \epsilon_r \epsilon_0$, $\epsilon_r = 11.7$ for Si) gives the first of the three coupled PDEs:

$$\nabla \cdot (\epsilon \nabla \psi) = -q(p - n + N_D^+ - N_A^-) \quad (2.1)$$

Interpretation: the curvature of ψ is the net charge density. Mobile charge (n, p) and fixed ionized dopants (C) both appear; n and p are *unknowns* in the coupled solver, and C is data. **[IMPLEMENTED]** in 1D, 2D and 3D.

2.1 What the equation predicts: the p-n junction

For an abrupt junction with $N_A = N_D = 10^{17} \text{ cm}^{-3}$ the depletion approximation (mobile carriers swept out of a layer of width W , leaving fixed ionic charge) integrates (2.1) twice:

$$V_{bi} = V_T \ln \frac{N_A N_D}{n_i^2}, \quad W = \sqrt{\frac{2\epsilon V_{bi}}{q} \left(\frac{1}{N_A} + \frac{1}{N_D} \right)}, \quad E_{max} = \frac{qNW}{2\epsilon} \quad (\text{symmetric}) \quad (2.2)$$

Numbers for this case — analytic: $V_{bi} = 0.8300$ V, $W = 0.1465 \text{ } \mu\text{m}$, $E_{max} = 1.13 \times 10^5$ V/cm. **The repository's own solver** (equilibrium solve on a 250-point graded mesh) returns $V_{bi} = 0.82999867$ V (matches the analytic value to 7 digits: Boltzmann statistics, BGN inactive since $N < N_0$) and a peak field of 1.098×10^5 V/cm (3% below the depletion estimate because real carrier tails smooth the edges).

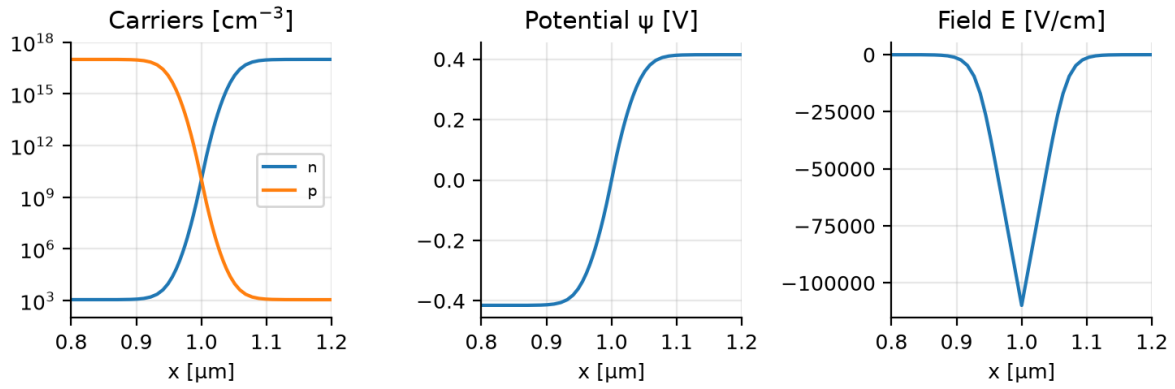


Figure 2 — Equilibrium n , p (log), ψ and E across a $10^{17}/10^{17}$ Si junction, computed with `Device1D.solve_equilibrium()` (generated for this guide by running the repo). Note the ~ 14 decades of minority density.

PHYSICAL INTUITION The Debye length is the natural ruler

Charge screening decays over $L_D = \sqrt{\epsilon V_T / (qN)}$. For $N = 10^{17} \text{ cm}^{-3}$, $L_D = 12.9 \text{ nm}$ (computed). The code non-dimensionalizes x by L_D so Poisson becomes $\psi'' = n - p - C$ with no stray constants — and it tells you how fine the mesh must be: $h \lesssim L_D$ near junctions (§7.2).

2.2 Equilibrium solve in the code

At $V=0$ the quasi-Fermi levels coincide, so carriers are *slaved* to ψ by (1.3) and the continuity equations are satisfied with zero current. The code solves a Poisson-only Newton problem:

`Device1D::solve_equilibrium_boltzmann` (core/src/device1d/device1d.cpp:572). Initial guess $\psi = \text{asinh}(C/2n_{ie})$ (the local charge-neutral solution), contacts pinned to $\psi_0 = \ln(n_0/n_{ie})$, then iterate $F_i = \tilde{\epsilon}_{i+1/2}(\psi_{i+1} - \psi_i)/h_i - \tilde{\epsilon}_{i-1/2}(\psi_i - \psi_{i-1})/h_{i-1} - dV_i(n_i - p_i - C_i) = 0$ with diagonal Jacobian entry $-\tilde{\epsilon}/h - \tilde{\epsilon}/h - dV(n+p)$. Variants: `solve_equilibrium_fd` (Fermi-Dirac, in η -space), `solve_equilibrium_dg` (density gradient).

[IMPLEMENTED]

2.3 Dielectrics and the MOS gate

Outside the semiconductor, Poisson holds with $\rho=0$ in the oxide ($\epsilon_{ox} = 3.9 \epsilon_0$, `EPS_OX_R` in `moscap.py`). Rather than meshing the oxide, the 2D/3D devices fold it into a **Robin boundary condition** on ψ at the silicon surface: oxide field $= (V_g - V_{fb} - \psi_s)/t_{ox}$, giving a boundary flux $\epsilon_{ox}(V_g - V_{fb} - \psi_s)/t_{ox}$ added to the surface node's Poisson row. Code: class `GateBC` ($\kappa = \epsilon_{ox} L_D / (\epsilon t_{ox})$) in `device2d.py:79` and `device3d.py:111` (with `normal_axis`); the stamp is $F += \kappa w (Vg_s - Vfb_s - (\psi_i - \psi_b))$, Jacobian $-\kappa w$.

[IMPLEMENTED]

Physics → equation → discretization → solver → code: electrostatics

Physics	Equation	Discretization	Solver	Source (file · class/function)
Gauss's law	$\nabla \cdot (\epsilon \nabla \psi) = -q(p - n + C)$	box integration: flux difference across a control volume	Newton; tridiagonal ψ -block (1D)	device1d.cpp residual_jacobian (Poisson row); solve_equilibrium_boltzmann
2D/3D Poisson	same, tensor-product stencil	5-point (2D) / 7-point (3D) box FV	Newton; sparse direct / Krylov	device2d.py _residual_jacobian_poisson :844; device3d.py:952; core/src/device3d/ poisson_eq.cpp
Gate / oxide	Robin: $\epsilon_{ox}(V_g - V_{fb} - \psi_s)/t_{ox}$	boundary-node source + diagonal term	same Newton	GateBC (device2d.py:79, device3d.py:111)
Permittivity steps	$\tilde{\epsilon}$ per edge	edge weight <code>et_</code>	n/a	device1d.cpp et_[] ; per-node <code>eps</code> arrays for heterostructures

3. Drift-diffusion transport and the continuity equations

3.1 Where current comes from

Carriers move by two mechanisms: **drift** in the electric field (velocity $v = \pm\mu E$) and **diffusion** down a density gradient (flux $-D\nabla n$). The Einstein relation $D = \mu kT/q$ ties them together, so one mobility μ per carrier is enough:

$$\mathbf{J}_n = q\mu_n n \mathbf{E} + qD_n \nabla n = -q\mu_n n \nabla \psi + q\mu_n V_T \nabla n, \quad \mathbf{J}_p = q\mu_p p \mathbf{E} - qD_p \nabla p \quad (3.1)$$

An equivalent and more illuminating form uses the **quasi-Fermi potentials** ϕ_n, ϕ_p defined by $n = n_{ie} e^{(\psi - \phi_n)/V_T}$:

$$\mathbf{J}_n = -q\mu_n n \nabla \phi_n, \quad \mathbf{J}_p = -q\mu_p p \nabla \phi_p \quad (3.2)$$

PHYSICAL INTUITION Current flows only where the quasi-Fermi level has a slope

In equilibrium $\phi_n = \phi_p = \text{const}$, so $\mathbf{J} = 0$ even though drift and diffusion are each enormous and cancel. Under bias the Fermi levels split (Figure 3). That cancellation is the numerical trap that Scharfetter-Gummel (§7.4) is designed to survive: the two terms in (3.1) are $\sim 10^{15} \times$ the net current near junctions.

3.2 Continuity (conservation of carriers)

$$\nabla \cdot \mathbf{J}_n = q(R - G) + q \frac{\partial n}{\partial t}, \quad \nabla \cdot \mathbf{J}_p = -q(R - G) - q \frac{\partial p}{\partial t} \quad (3.3)$$

Steady state drops $\partial/\partial t$; transient ([**IMPLEMENTED**], §8.2) restores it. R is the net recombination rate (§4) and G external/impact generation. The three equations (2.1), (3.1) and (3.3) with unknowns (ψ, n, p) are the **van Roosbroeck system**, solved in the code as 3 degrees of freedom per node, ordered ψ, n, p within each node ($F[3*i]$ Poisson, $F[3*i+1]$ electron continuity, $F[3*i+2]$ hole continuity in `residual_jacobian`).

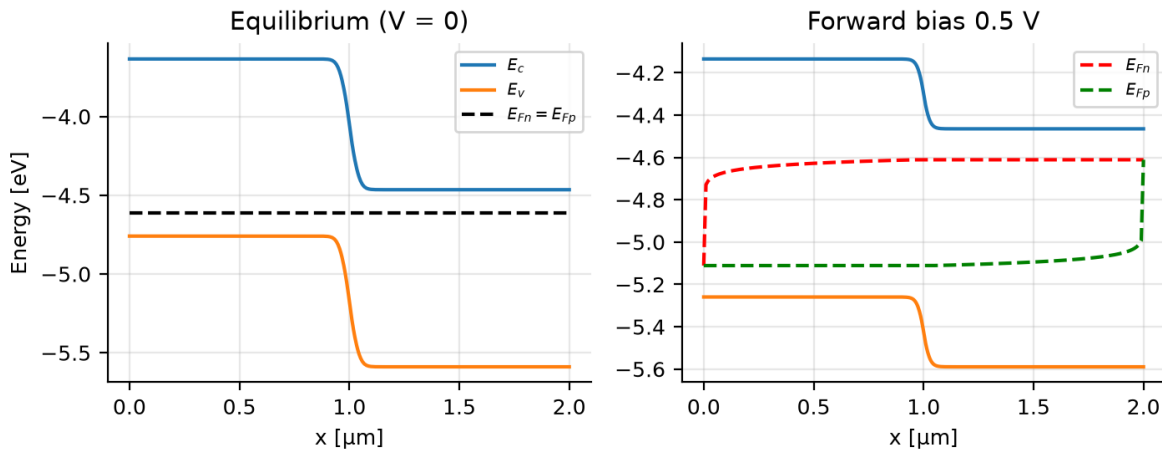


Figure 3 — Band diagram of the $10^{17}/10^{17}$ Si diode at equilibrium (flat E_F) and at 0.5 V forward bias (E_{Fn}, E_{Fp} split by qV across the junction), from `Device1D.band_diagram()`. Quasi-Fermi levels are flat in the quasi-neutral majority regions and drop only where minority carriers recombine.

3.3 Static device physics: the ideal diode

Integrating (3.3) over the neutral regions with low injection gives the Shockley relation. For the short-base diode used in the repository's tests the saturation current scales with minority density and diffusivity over the base width W_b :

$$J = J_s (e^{V/V_T} - 1), \quad J_s \approx q \left(\frac{D_p p_{n0}}{W_n} + \frac{D_n n_{p0}}{W_p} \right) \quad (3.4)$$

The repository's README records $J(0.5 \text{ V}) = 1.280 \times 10^{-2} \text{ A/cm}^2$ for the abrupt $10^{17}/10^{17}$ Si diode versus 1.321×10^{-2} from the short-base formula; **re-running** the solver for this guide gave $J(0.5 \text{ V}) = \mathbf{1.2801 \times 10^{-2} A/cm^2}$. The $\sim 3\%$ gap is the physics the ideal formula omits (SRH recombination in the depletion region, Auger, finite injection, bandgap narrowing). [**IMPLEMENTED**]

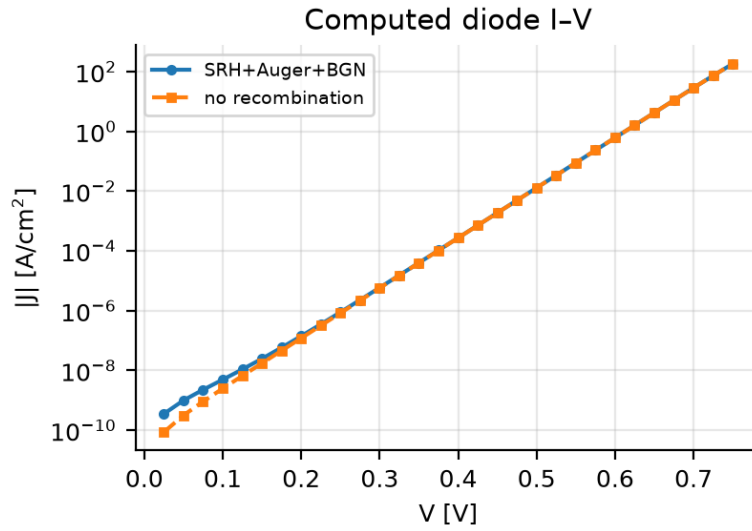


Figure 4 — Computed forward I-V from `Device1D.iv_sweep()` with and without SRH+Auger recombination. The two curves nearly coincide on this scale. The straight region has slope $\approx q/(\eta kT)$; the ideality factor is checked by `test_g3_ideality_factor` (`tests/test_device1d_native_gates.py`).

Physics → equation → discretization → solver → code: drift-diffusion transport

Physics	Equation	Discretization	Solver	Source (file · class/function)
Carrier currents	$J_n = q\mu_n(nE + V_T\nabla n)$ (3.1)	edge Scharfetter-Gummel: $J_{n,k} = a_{n,k+1}(n_{k+1}^{B(\delta)} - n_k^{B(-\delta)})$	evaluated inside Newton assembly	device1d.cpp residual_jacobian ($J_n[k], J_p[k]$); physics/kernels.hpp bernoulli ; kernels.py:88
Einstein relation	$D = \mu V_T$	edge D = harmonic mean of $\mu \times V_T/D0_REF$	n/a	inputs.cpp dn_edge, dp_edge ; device1d.cpp hmean
Continuity	$\nabla \cdot J_n = q R$	$F_{n,i} = J_R - J_L - R_i dV_i$	Newton rows 3i+1, 3i+2	device1d.cpp residual_jacobian
Terminal current	$I = J$ at contact edge	edge current, conservation checked	post-processing	device.py current_density ; device2d.py terminal_current :2194; device3d.py:2274

4. Generation, recombination and mobility

4.1 Recombination

Recombination returns a perturbed np product toward n_{ie}^2 . The repository implements three channels in the baseline path and several more as options.

Shockley-Read-Hall (trap-mediated) [IMPLEMENTED]

$$R_{SRH} = \frac{np - n_{ie}^2}{\tau_p(n + n_{ie}) + \tau_n(p + n_{ie})} \quad (4.1)$$

Mid-gap traps ($n_1=p_1=n_{ie}$). Lifetimes: Scharfetter fit $\tau = \tau_0/(1+N/N_{ref})$ with $\tau_{n0}=10 \mu s$, $\tau_{p0}=3 \mu s$, $N_{ref}=5 \times 10^{16}$ (`materials.lifetime_scharfetter`). Function: `materials.recombination` (Python reference) / `tcad::physics::recombination_boltzmann` (C++, physics/kernels.hpp:58) which also returns $\partial R/\partial n$ and $\partial R/\partial p$ for the Jacobian.

Auger [IMPLEMENTED]

$$R_{Aug} = (C_n n + C_p p)(np - n_{ie}^2), \quad C_n = 2.8 \times 10^{-31}, \quad C_p = 9.9 \times 10^{-32} \text{ cm}^6/\text{s} \quad (4.2)$$

Three-particle process, dominant at high injection / heavy doping. Default on (`Models(auger=True)`).

Trap-assisted tunneling (Hurkx) [IMPLEMENTED]

Models(tat=True): field-enhanced SRH. The capture denominators gain WKB escape probabilities P_n, P_p : $R_{TAT} = (np - n_{ie}^2) / [\tau_p(n + n_{ie}(1 + P_p)) + \tau_n(p + n_{ie}(1 + P_n))]$ (visible verbatim in the C++ assembly, "R_TAT = ..."). Probabilities are *frozen per bias solve* (tat_probabilities). Zero probabilities reproduce plain SRH bit-for-bit (tested: test_g_tat_zero_probabilities_are_plain_srh). Documented gotcha: WKB factors are SI (F in V/m); V/cm underflows to plain SRH.

Other recombination/generation channels

- Surface recombination at contacts — Robin BC $J \cdot \hat{n} = qS(n - n_0)$ (**Models(S_n, S_p)**) [PARTIALLY IMPLEMENTED]: Device1D and Device2D; **Device3D refuses** (device3d.py raises).
- Radiative (band-to-band optical) recombination [NOT IMPLEMENTED]: no radiative term exists in the code (searched).
- Photo-generation / optics [NOT IMPLEMENTED] (M40 optics not started).
- Impact-ionization generation $G = (\alpha_n |J_n| + \alpha_p |J_p|) / q$ (§10.1) [IMPLEMENTED]; band-to-band tunneling generation (§10.2) [IMPLEMENTED].

4.2 Mobility

Mobility μ measures how easily a carrier is accelerated between scattering events. It is *the* empirical input to drift-diffusion; each scattering mechanism (phonons, ionized impurities, surface roughness, velocity saturation) reduces it. PyTCAD layers four models:

(a) Doping dependence — Caughey-Thomas [IMPLEMENTED]: `materials.mobility_caughey_thomas`.

$$\mu(N) = \mu_{min} + \frac{\mu_{max} - \mu_{min}}{1 + (N/N_{ref})^\alpha}, \quad \mu^n: (92, 1360, 1.3 \times 10^{17}, 0.91) \quad (4.3)$$

Si electron ($\mu_{min}, \mu_{max}, N_{ref}, \alpha$) as shown; holes (47.7, 495, 6.3×10^{16} , 0.76). μ_{max} carries a temperature exponent $(T/300)^{-2.33}$ (n), $^{-2.23}$ (p). **Critical gotcha**: N is the *total* ionized impurity $N_A + N_D$, not $|N_D - N_A|$; for compensated regions pass $N_{total} =$ to the device constructor (example in README).

(b) High-field velocity saturation — Canali [PARTIALLY IMPLEMENTED]: $\mu(E) = \mu_0 / [1 + (\mu_0 |E| / v_{sat})^\beta]^{1/\beta}$ with $v_{sat} = 1.07 \times 10^7$ (n), 8.37×10^6 cm/s (p), $\beta = 2$ (n), 1 (p) (`materials.mobility_field`). Applied **lagged** inside Newton (frozen at the previous iterate, which costs quadratic convergence). Available only in **Device1D** (**Models(field_mobility=True)**); Device2D/3D refuse the flag.

(c) Surface (inversion-layer) mobility — Lombardi CVT [PARTIALLY IMPLEMENTED]: `materials.mobility_cvt`; **Models(surface_mobility=True)** is applied lagged on **2D devices with a gate**; 1D and 3D raise. Absolute calibration (M14 gate G-A) is **open**: the Lombardi 1988 constants are paywalled, so only trends and the COMSOL-comparison δ are gated.

(d) Energy-balance mobility [PARTIALLY IMPLEMENTED]: with `energy_balance=True` the effective field comes from the carrier temperature (§10.5) and replaces (b).

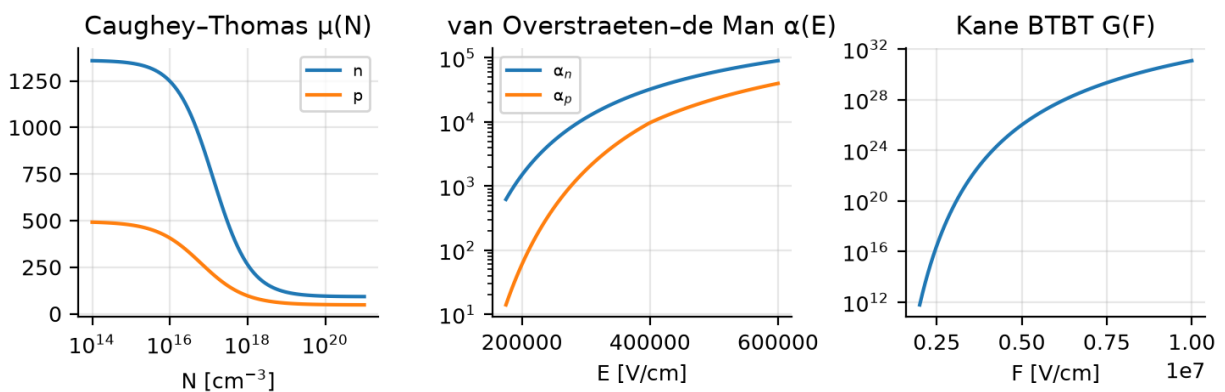


Figure 5 — Left: Caughey-Thomas mobility vs total doping (Si, 300 K). Centre: van Overstraeten-de Man impact-ionization coefficients (§10.1). Right: Kane BTBT generation vs field (§10.2). All three evaluated by calling the repository functions.

NUMERICAL IMPLICATION Why mobility is lagged

μ depends on E , which depends on ψ . Differentiating $\mu(\psi)$ analytically would add dense-ish Jacobian columns. The code instead freezes μ at the last iterate (`set_edge_diffusivity` every Newton pass, `device1d.cpp:277`). Consequence: convergence degrades from quadratic to linear near the saturation knee, but the Jacobian stays an exact derivative of the *frozen- μ* residual, which is what the finite-difference gates check.

Physics → equation → discretization → solver → code: recombination and mobility

Physics	Equation	Discretization	Solver	Source (file · class/function)
SRH, Auger	(4.1),(4.2)	pointwise at each node, scaled by R_0	analytic $\partial R/\partial n, \partial p$ in Jacobian	materials.py recombination ; physics/kernels.hpp recombination_boltzmann ; device1d.cpp (<code>Rs[j]</code>)
TAT	Hurkx with WKB $P_{n,p}$	frozen P per solve	same	device1d.cpp tat_probabilities ; tests/test_m12_tat.py
Doping mobility	Caughey-Thomas	node μ , edge harmonic mean	n/a	materials.py mobility_caughey_thomas ; inputs.cpp
Field mobility	Canali	lagged edge μ	Newton outer lag	materials.py mobility_field ; device1d.cpp mobility_field :287
Surface mobility	Lombardi CVT	lagged, per gate-adjacent node (2D)	Newton outer lag	materials.py mobility_cvt ; device2d.py _update_surface_mobility :612

5. Boundary conditions and contacts

A PDE needs boundary conditions at every boundary. In a device, three kinds appear: **ohmic contacts** (Dirichlet), **insulating boundaries** (natural Neumann), and **gate/Schottky contacts** (Robin-type).

5.1 Ohmic contact (Dirichlet)

An ideal ohmic contact is an infinite sink/source of carriers: the semiconductor next to it is in local charge-neutral equilibrium with the metal. So the carriers are pinned to their neutral values, and the potential is the applied bias plus the built-in offset:

$$n = n_0, \quad p = p_0 \quad (\text{eq. 1.5}), \quad \psi = \frac{V}{V_T} + \ln \frac{n_0}{n_{ie}} \quad (5.1)$$

[IMPLEMENTED] Code: `Device1D::contact_value(side, V)` (`device1d.cpp:301`) returns $\{\psi_0, n_0, p_0\}$ (a Fermi-Dirac / incomplete-ionization variant solves neutrality in η -space via `fd_neutral_eta`). The heterojunction/affinity gauge shifts ψ_0 by the band shift s (M33). In 2D/3D: classes `DirichletBC` (`add_contact`), 3D also `PinnedBC`. Contacts are *not* meshed regions; they are sets of boundary nodes.

5.2 Insulating boundaries and symmetry planes

Everywhere else, the natural finite-volume boundary condition applies: the flux through the boundary face is zero ($\mathbf{j} \cdot \hat{\mathbf{n}} = 0$ and $\epsilon \partial \psi / \partial n = 0$) because the control volume simply has no neighbour there. Nothing is coded — the *absence* of an edge term *is* the Neumann condition. **[IMPLEMENTED]**

5.3 Gate (Robin) and Schottky contacts

Gate **[IMPLEMENTED]**: see §2.3 — an oxide-capacitance Robin term on ψ only (no carrier flux into the oxide). **Schottky** **[PARTIALLY IMPLEMENTED]**: a metal-semiconductor interface has a barrier $\Phi_B = \Phi_M - \chi$ and is not ohmic. Two modes (class `SchottkyContact` in 1D, `SchottkyBC` in 2D/3D): **Dirichlet** — pin the majority density at its barrier-limited equilibrium value; **Robin / thermionic** — the surface current is limited by emission velocity $v_R = A^* T^2 / (q N_c)$: $J = q v_R (n - n_0)$. The Robin mode reuses the surface-recombination machinery. Support: Dirichlet + Robin in 1D and 2D; **3D Dirichlet only** (ARCHITECTURE.md capability matrix). The standalone analytic module `pytcad/schottky.py` gives Richardson constant (`richardson_a_star`), image-force lowering, thermionic I-V (`schottky_iv`) and field-emission regime classification (`contact_regime`) used as published-value gates (`tests/test_m28_schottky.py`, `test_m46_*`).

5.4 How a Dirichlet row is imposed numerically

Naïvely replacing a Jacobian row by the identity makes the matrix non-symmetric in its constrained structure and breaks adjoint solves. The code uses **symmetric elimination** (`pytcad/dirichlet.py`, C++ twin `eliminate_dirichlet` in `device1d.cpp`): constrained unknowns are substituted out, $J_{ik}g_k$ moves to the right-hand side, and the constrained columns are emptied. The solution is identical in exact arithmetic; only the floating-point path changes (documented module docstring). Tests: `tests/test_dirichlet_elimination.py`.

Physics → equation → discretization → solver → code: contacts and boundaries

Physics	Equation	Discretization	Solver	Source (file · class/function)
Ohmic contact	$n=n_0, p=p_0, \psi=V/V_T + \ln(n_0/n_{ie})$	Dirichlet DOFs	symmetric elimination	<code>device1d.cpp</code> contact_value :301; eliminate_dirichlet ; <code>dirichlet.py</code> eliminate_coo ; <code>device2d.py</code> DirichletBC :70, _bc_contact_values :689
Gate	Robin on ψ	boundary node weight w	Newton diagonal term	GateBC (2D:79, 3D:111)
Schottky	Φ_B , Richardson v_R	Dirichlet or Robin	Newton	<code>device.py</code> SchottkyContact :631; <code>device2d.py</code> SchottkyBC :91; <code>schottky.py</code>
Surface recomb.	$J \cdot \hat{n} = qS(n-n_0)$	Robin term	Newton	Models.S_n, S_p ; <code>device2d.py</code> (Robin block)
Insulating edge	$J \cdot \hat{n}=0, \partial\psi/\partial n=0$	omit boundary edge	none	implicit in all assemblies

6. Devices: the physics that actually appears in the repo

6.1 P-N junction and diode

[IMPLEMENTED] (1D, 2D, 3D). The built-in potential and depletion width (§2.1) set the electrostatics; forward bias lowers the barrier $V_{bi}-V$ and exponentially increases minority injection; reverse bias widens W and leaves only the saturation/generation current until breakdown (§11). Example: `examples/01_pn_diode.py`. Tested by FD-Jacobian, built-in-potential, forward-current, mesh-refinement and ideality gates (`tests/test_deviceld_native_gates.py`: G0-G5).

6.2 MOS capacitor

[IMPLEMENTED] A metal/poly gate, oxide and silicon substrate. Gate bias changes surface potential ψ_s : **accumulation** (majority carriers pile up), **depletion** (majority carriers pushed out, fixed charge exposed), **inversion** (minority carriers collect, $\psi_s > 2\phi_F$ for the threshold condition). The class `MOSCapacitor` (`pytcad/moscap.py`:93) solves a 1D nonlinear Poisson equation through the silicon with a gate Robin boundary:

$$V_g = V_{fb} + \psi_s + \frac{Q_s}{C_{ox}}, \quad V_{fb} = \Phi_{MS} - \frac{Q_f}{C_{ox}}, \quad C_{ox} = \frac{\epsilon_{ox}}{t_{ox}} \quad (6.1)$$

Gate options `n+poly`, `p+poly`, `Al` or a numeric work function; fixed oxide charge Q_f ; interface traps D_{it} (M14: $C_{it}=q^2D_{it}$); optional Fermi-Dirac and density-gradient quantum correction. Entry points: `solve_psi(Vg)` (Newton, :204), `cv_sweep(Vg_list)` (returns ψ_s , Q_g , C_g : :609), `flatband_voltage`, `analytic_landmarks`, `inversion_centroid`. **Quasi-static** C-V only (no high-frequency minority-carrier freeze-out solve beyond the gated QS/HF comparison test). Gates in `tests/test_cv_physics_validation.py`: discrete Poisson residual zero, global charge neutrality, Gauss surface balance, regime ordering, series-capacitance doping recovery, C_{min} match, V_{fb} crossing, $\psi_s=2\phi_F$ at threshold, D_{it} stretch-out.

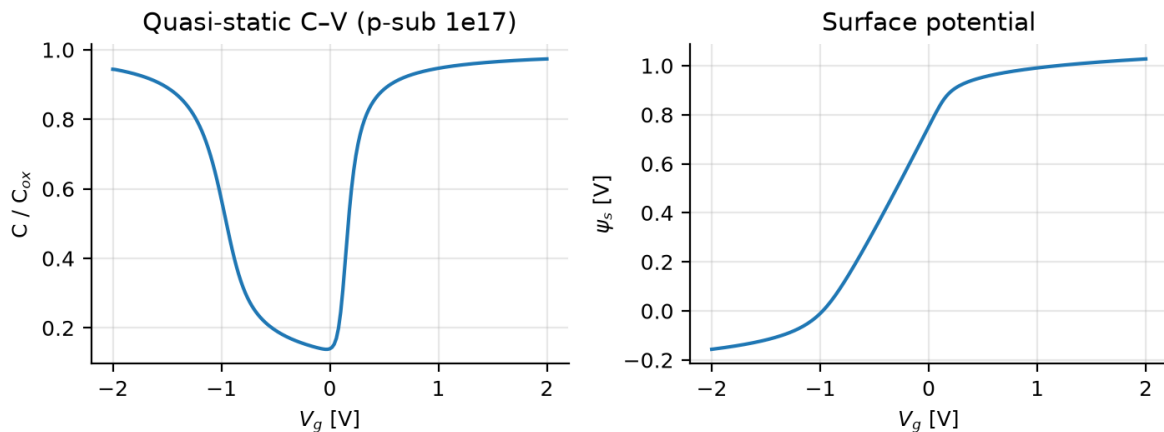


Figure 6 — Quasi-static C-V and surface potential of an n^+ -poly / 5 nm oxide / p -Si (10^{17}) MOS capacitor from `MOSCapacitor.cv_sweep()`. Accumulation (left, $C \rightarrow C_{ox}$), depletion minimum near 0 V, low-frequency inversion rebound (right). ψ_s pins near $2\phi_F \approx 0.8$ V as the inversion layer screens the gate.

6.3 MOSFET and multi-gate devices (2D and 3D)

[IMPLEMENTED] `Device2D` adds a gate Robin BC, source/drain ohmic contacts and surface mobility; I_d - V_g sweeps in `examples/04_mosfet_idvg.py` (`pytcad/mosfet.py`). `Device3D` lifts this to a 3D box mesh with 3D gates (normal_axis) enabling FinFET / gate-all-around studies (`finfet3d.py`, `gmsh_finfet3d.py`, `umos3d.py`, `sic_vmosfet.py`). The central **reduction gate** principle: a uniform 3D case must reproduce the validated 2D (and 1D) answer to round-off (ARCHITECTURE.md rule); see `examples/05_3d_reduces_to_2d.py`. Short-channel metrics DIBL and subthreshold swing are tested in `test_finfet3d_dibl_and_subthreshold_swing_worsen_as_gate_length_shrinks`. Quantum confinement in thin bodies is represented by density-gradient (§10.4), equilibrium-only **[PARTIALLY IMPLEMENTED]**.

6.4 Heterojunctions, HBT, HEMT

[IMPLEMENTED] Per-node material arrays (Si, Ge, GaAs, InGaAs, AlGaAs via workbench library; `materials.GE, GAAS, INGAAS, algaas(x)`). The central physics: at an interface the band edges shift by χ and E_g steps (Anderson rule). Two gauges (`Models(band_offset=...)`): **'nie'** (legacy) encodes only N_c, N_v, E_g in n_{ie} and therefore splits any offset *symmetrically* between E_c and E_v — the repo measured that a 0.5 eV χ step changed the solution by **exactly zero** in this gauge; **'affinity'** uses true band edges. Electron SG deltas gain $+d\ln n_{ie}$, hole deltas $-d\ln n_{ie}$ (a shared delta passes FD-Jacobian but breaks hole detailed balance — a recorded gotcha). `Models(thermionic=True)` replaces SG on abrupt interfaces by a detailed-balanced thermionic flux (requires 'affinity'; refused with impact ionization). Workbench templates: diode, MOS-C, NMOS, HBT, HEMT, FinFET (`core/src/uicore/templates.cpp`, `workbench/core/templates.py`).

6.5 Schottky diode

[PARTIALLY IMPLEMENTED] (see §5.3) `examples/11_schottky_diode.py`; thermionic-emission I-V matches theory at a published barrier; barrier lowering grows with field (tests in `test_model_benchmarks.py`).

6.6 Power and other structures present in the repo

3D SiC power MOSFET (`examples/07_3d_sic_power_mosfet.py`, `sic_vmosfet.py`), U-MOSFET with adaptive mesh refinement (`umos3d.py`, `examples/08_3d_umos_amr.py`), LOCOS process flow (`08_locos_flow.py`). Avalanche and breakdown (§10.1) and self-heating (§8.4) are the key physical extensions for power devices. **[IMPLEMENTED]** for the structures, **[PARTIALLY IMPLEMENTED]** for breakdown-voltage extraction (open gate G-D).

6.7 Mixed-mode circuits

[IMPLEMENTED] `pytcad/circuit.py`: SPICE-style Modified Nodal Analysis with resistors, capacitors, V/I sources, exponential diode, a level-1 square-law MOSFET (no body effect, hard cutoff) and a `DeviceStamp` embedding a real Device1D/2D/3D as a nonlinear element. **Disclosed honestly in the module docstring:** the stamp's small-signal conductance is a finite-difference “companion model” (two full solves per Newton iteration), not the literal reuse of the device Jacobian.

Physics → equation → discretization → solver → code: devices

Physics	Equation	Discretization	Solver	Source (file · class/function)
PN diode	(2.1),(3.1),(3.3)	1D box FV, graded mesh	Newton, block-tridiagonal LU	<code>device.py</code> Device1D ; <code>examples/01_pn_diode.py</code>
MOS-C	1D Poisson + gate Robin	box FV through Si	scalar Newton (tridiagonal)	<code>moscap.py</code> MOSCapacitor
MOSFET / FinFET	2D/3D DD + gate Robin	tensor-product box FV	Newton + sparse direct/Krylov	<code>device2d.py</code> , <code>device3d.py</code> , <code>mosfet.py</code> , <code>finfet3d.py</code>
Heterojunction	Anderson offsets	per-node n_{ie} , χ ; carrier-specific SG deltas	same	<code>device1d.cpp</code> (<code>band_shift_delta/delta_p</code>); <code>Models.band_offset</code>
Circuit	KCL (MNA) + device	companion model	outer Newton	<code>circuit.py</code> DeviceStamp

7. From equations to a discrete system

7.1 Scaling (non-dimensionalization)

Raw quantities span absurd ranges (n from 10^3 to 10^{20} cm^{-3} , E from 0 to 10^6 V/cm). The code scales every unknown to $O(1)$:

$$\hat{\psi} = \frac{\psi}{V_T}, \quad \hat{n} = \frac{n}{N_s}, \quad \hat{x} = \frac{x}{L_D}, \quad L_D = \sqrt{\frac{\epsilon V_T}{q N_s}}, \quad J_0 = \frac{q D_0 N_s}{L_D}, \quad R_0 = \frac{D_0 N_s}{L_D^2} \quad (7.1)$$

with $D_0=1$ cm^2/s (`D0_REF`). Poisson then reads $\hat{\psi}'' = \hat{n} - \hat{p} - \hat{C}$. **N_s is the peak doping, not n_i** (`r.Ns = max(dmax, ni)`, `core/src/device1d/inputs.cpp:54-57`; scaled x is computed there too). Pitfall recorded in `CLAUDE.md`: `Device3D` takes N_s from `max|doping|` of the array it was built with, so subdomain devices must share an `Ns_override` computed from the full device. **[IMPLEMENTED]**

NUMERICAL IMPLICATION Scaling decides conditioning

Because $\hat{n}$ is $O(1)$ in doped regions but 10^{-7} in the minority tail ($10^{10}/10^{17}$), a relative convergence test on n is used (below), and updates are clipped multiplicatively. Poor scaling would make the Jacobian entries differ by 15+ orders of magnitude and ruin any direct factorization.

7.2 Meshing

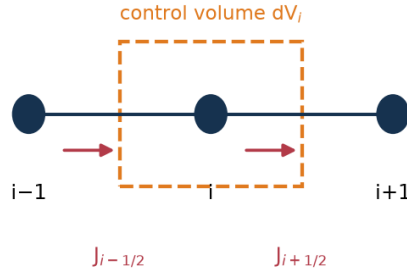
The Debye length is the resolution criterion: spacing $h \leq L_D$ wherever the potential changes (junctions, oxide interfaces, channels). Tools: `mesh.graded_mesh(L, x_focus, h_min, h_max, ratio=1.15)` (geometric grading toward focus points), `uniform_mesh`, `merge_mesh`, `debye_length`, and `check_mesh(x, doping)` which prints the worst h/L_D . For the diode in Figure 2: 250 nodes, $h_{\min}=10.5$ nm vs $L_D=12.9$ nm (computed).

[IMPLEMENTED]

Mesh type	Class / module	Dim.	Status
Non-uniform 1D line	<code>graded_mesh</code> → array x	1D	[IMPLEMENTED]
Tensor-product rectangle	<code>Mesh2D(x, y)</code> (<code>mesh2d.py:31</code>); <code>control_volume_widths</code>	2D	[IMPLEMENTED]
Tensor-product box	<code>Mesh3D(x, y, z)</code> (<code>mesh3d.py:20</code>)	3D	[IMPLEMENTED]
Unstructured triangles	<code>gmsh_mesh.py</code> , <code>unstructured_assembly.py</code> , <code>unstructured_dd.py</code>	2D	[IMPLEMENTED] (homojunction + hetero via per-node materials; some Models refused)
Unstructured tetrahedra	<code>gmsh_mesh3d.py</code> , <code>unstructured_assembly3d.py</code> , <code>unstructured_dd3d.py</code> ; C++ <code>core/src/unstructured3d</code>	3D	[IMPLEMENTED] (homojunction; hetero in M47-S3)
Adaptive refinement (AMR)	<code>adapt.py</code> , <code>adapt_unstructured.py</code> , <code>adapt_unstructured3d.py</code> ; C++ <code>process/indicators.cpp</code>	1D/2D/3D	[IMPLEMENTED]

7.3 Finite volume (box integration) — not finite element

The repository discretizes with **box integration** (a finite-volume method on node-centred control volumes): each node i owns a cell of volume dV_i ; integrate each PDE over the cell, apply the divergence theorem, and the divergence terms become *differences of fluxes through the cell faces*. This guarantees discrete conservation: current leaving one cell enters its neighbour. Finite-*element* assembly (basis functions, weak form) is not used.



$$dV_i = (h_{i-1} + h_i)/2 \quad ; \quad F_i = J_{i+1/2} - J_{i-1/2} - R_i dV_i$$

Figure 7 — 1D control volume. The scaled residual of the electron row is $F_i = J_{i+1/2} - J_{i-1/2} - R_i dV_i$ (verbatim in `device1d.cpp`: `Jn[eR] - Jn[eL] - Rs[i]*dV_[i]`).

In 2D the cell is $dV_{j,i} = dV_{y,j} \cdot dV_{x,i}$ (`np.outer(dVy, dVx)`, `mesh2d.py`); edge fluxes are scattered with the transverse dual width. On an unstructured triangle mesh the cell is the **dual (Voronoi-like) area**, built with the mixed Voronoi/barycentric method of Meyer et al. 2003 (`unstructured_assembly.py` docstring), and every edge carries a geometry factor $trans = dual\text{-}facet\text{-}length / primal\text{-}edge\text{-}length$ shared by the Poisson and SG terms. FEM **[NOT IMPLEMENTED]**; FV **[IMPLEMENTED]**.

7.4 Scharfetter-Gummel: the key idea

Discretizing (3.1) with a plain central difference fails: on a coarse mesh with a large potential step the scheme produces negative densities and wild oscillation. Scharfetter and Gummel (1969) instead *solve the 1D current equation exactly on each edge*, assuming J and the field are constant across the edge. With $\delta = (\psi_{k+1} - \psi_k)/V_T$ in scaled form:

$$J_{n,k+\frac{1}{2}} = \frac{\mu_n V_T}{h_k} [n_{k+1} B(\delta_k) - n_k B(-\delta_k)], \quad B(x) = \frac{x}{e^x - 1} \quad (7.2)$$

$$J_{p,k+\frac{1}{2}} = -\frac{\mu_p V_T}{h_k} [p_{k+1} B(-\delta_k) - p_k B(\delta_k)] \quad (7.3)$$

Verbatim in the code: `Jn[k] = an[k]*(n[k+1]*Bp[k] - n[k]*Bm[k]); Jp[k] = -ap[k]*(p[k+1]*Bm_h[k] - p[k]*Bp_h[k])` where `Bp=B(δ)`, `Bm=B(-δ)` and `an = dn_edge/h`. B has the two limits you must internalize: $\delta \rightarrow 0$, $B \rightarrow 1$ (pure diffusion, $J = qD \Delta n/h$); $\delta \rightarrow \pm\infty$, $B \rightarrow 0$ or $|\delta|$ (pure drift with the *upwind* density). That automatic upwinding is why SG is positivity-preserving and robust on coarse meshes. **[IMPLEMENTED]**

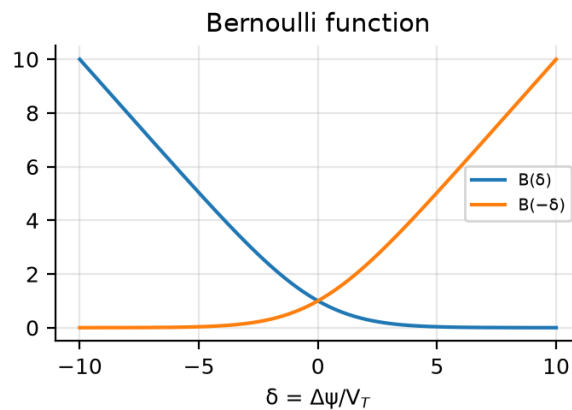


Figure 8 — Bernoulli function $B(\delta)$ and $B(-\delta)$ from `pytcad.kernels.bernoulli`. Note $B(\delta) - B(-\delta) = -\delta$, the identity that makes SG exact for constant field.

NUMERICAL IMPLICATION Numerically stable Bernoulli

$x/(e^x - 1)$ overflows for large $|x|$ and cancels for small $|x|$. The repo clips $|x| \leq 700$, uses the series $1 - x/2 + x^2/12$ for $|x| < 10^{-4}$ and `expm1` otherwise (`kernels.py:88` and `C++ physics/kernels.hpp:27`); dB/dx is formed as $B(1/x - 1/(1 - e^{-x}))$ with the series $-1/2 + x/6 - x^3/180$ near 0. The derivative feeds the exact Jacobian.

Extensions to SG already in the code: (i) **Fermi-Dirac**: the v -factor generalized SG adds $\Delta(L_n)$ to the electron δ and $-\Delta(L_p)$ to the hole δ , with $L = \ln(\text{density ratio factors})$ (`fd_node_factors`, `device.py:225`); (ii)

heterostructures: carrier-specific $\pm \Delta \ln n_{ie}$ plus the shared affinity shift Δs (§6.4); (iii) **thermionic edges:** SG replaced by detailed-balanced emission flux $K[n_2 g_2 - n_1 g_1]$ (device1d.cpp, if (models_.thermionic && ex_.te_edge[k])).

Physics → equation → discretization → solver → code: discretization

Physics	Equation	Discretization	Solver	Source (file · class/function)
Conservation law	$\nabla \cdot J = qR$	box integration over dV_i	—	device1d.cpp residual_jacobian ; mesh2d.py Mesh2D.dV
Drift+diffusion on an edge	(3.1), J const	Scharfetter–Gummel	exact derivative via dB/dx	physics/kernels.hpp bernoulli/dbernoulli ; kernels.py:88,99
Unstructured geometry	dual-cell flux	trans = facet/edge length	—	unstructured_assembly.py build_edge_flux_geometry
Fermi–Dirac current	v -factor SG	Δ_L in δ	extra w -columns in J	device.py fd_node_factors :225; device1d.cpp (L_n, L_p, w_n, w_p)

7.5 The Jacobian

Newton's method needs $J = \partial F / \partial u$ for $u = (\psi, n, p)$. Because each residual row couples only a node and its neighbours, J is sparse. The repository uses **analytic** Jacobians (never automatic differentiation), validated by **finite-difference gates** to 5×10^{-5} (README). Example, the electron row at an interior node i exactly as stamped in device1d.cpp:

$$\frac{\partial F_{n,i}}{\partial n_i} = -a_n^R B_-^R - a_n^L B_+^L - \frac{\partial R_i}{\partial n_i} dV_i, \quad \frac{\partial F_{n,i}}{\partial n_{i \pm 1}} = a_n B(\pm \delta), \quad \frac{\partial F_{n,i}}{\partial \psi_{i \pm 1}} = \frac{\partial J_n}{\partial \psi} \quad (7.4)$$

The cross-couplings ($\partial R / \partial p$, $\partial J / \partial \psi$) are what make the system *fully coupled*: a change of ψ moves both currents and the Poisson charge. The alternative — Gummel decoupling (solve Poisson, then continuity, iterate) — is cheaper per step but converges linearly and fails at high injection; the repo's choices: **fully coupled Newton** for the electrical system, and an outer **Gummel-style loop only for self-heating** (§8.4).

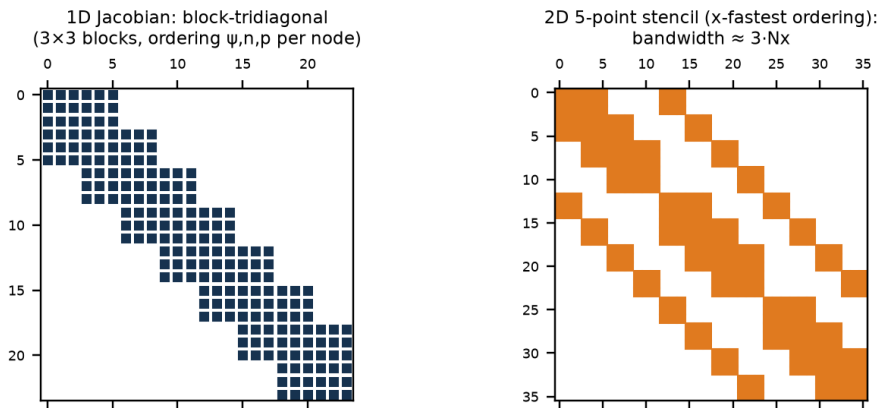


Figure 9 — Sparsity patterns. 1D: block-tridiagonal with 3×3 blocks (ψ, n, p interleaved per node). 2D: a 5-point stencil of 3×3 blocks; bandwidth grows with the grid width, which is why 2D/3D need good sparse direct solvers or Krylov methods (§7.7).

CAUTION Round-off in near-zero edge currents

ARCHITECTURE.md §6 documents a real hazard: near a junction an edge current can equal the difference of two terms $\sim 10^{15} \times$ larger, so its sign and size are pure round-off. Anything that depends on $|J|$ (the impact-ionization source) then differs between builds. This is the open M15 G-D item (§10.1). The lesson for learners: SG currents are accurate in *drift-diffusion balance*, but derived quantities that amplify the near-cancelling difference are fragile.

7.6 Newton-Raphson with damping

Newton-Raphson loop (`Device1D::newton` in `core/src/device1d/device1d.cpp`)

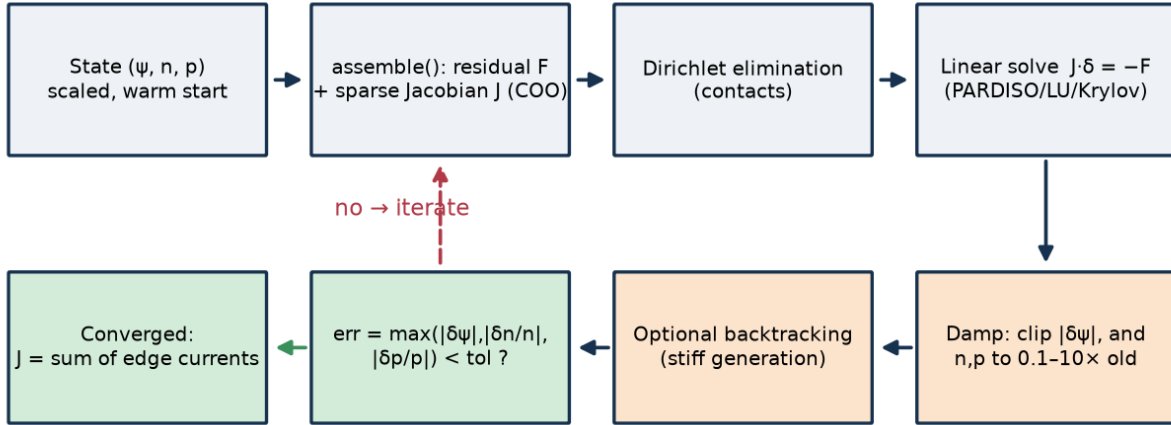


Figure 10 — The Newton loop as implemented by `Device1D::newton` (`device1d.cpp:1542`). The Python 2D/3D drivers follow the same structure.

Algorithm. At iterate u^k : assemble F and J ; solve $J \delta = -F$; update. Convergence is judged on the *update*, not the residual (README): the residual's scale mixes Poisson and continuity rows, whereas the scaled update directly measures remaining change in ψ , $\ln n$, $\ln p$.

$$\text{err} = \max\left(\max_i |\delta\psi_i|, \max_i \left|\frac{n_i^{\text{new}}}{n_i} - 1\right|, \max_i \left|\frac{p_i^{\text{new}}}{p_i} - 1\right|\right) < \text{tol_update} = 10^{-8} \quad (7.5)$$

Damping/globalization in the code: (1) clip $|\delta\psi| \leq \text{max_dpsi}$ (default 5 scaled units = $5V_T$); (2) clip n, p updates to $[0.1\times, 10\times]$ of the old value so densities stay positive; (3) for stiff generation models (impact, BTBT) a **backtracking line search** on the merit $\frac{1}{2}\|F\|^2$ (up to 10 halvings, active while $\text{err} \geq 10^{-3}$, accepts when merit decreases by $10^{-4}\lambda$); (4) a **generation-strength ladder** `kIiStages` = [0, 0.02, 0.05, 0.1, 0.2, 0.35, 0.5, 0.7, 1.0] multiplies the generation source and re-solves each stage warm-started, stopping at the first failure; (5) **bias continuation** and **arc-length** sweeps (§8.1). **[IMPLEMENTED]**

PHYSICAL INTUITION Why Newton works here — and where it fails

Near the solution the system is smooth and Newton converges quadratically. Far from it the exponentials ($n \propto e^\psi$) make F extremely nonlinear; a 1 V step in forward bias changes n by e^{39} . Warm-starting from the previous bias point plus ψ -clipping tame it. Breakdown (avalanche) creates a genuine *fold* (turning point) in the I-V curve where J becomes singular: only arc-length continuation can pass it.

7.7 Sparse direct and iterative solvers; PETSc

Each Newton step solves $J \delta = -F$. Implemented backends (`pytcad/linsolve.py:solve_linear`, methods `direct`, `gmres`, `bicgstab`, `gpu_direct`, `petsc`, `mumps`):

Backend	What it is	Where	Status
Eigen SparseLU	C++ sparse LU; symbolic analysis reused while the pattern is unchanged	<code>core/src/solver/direct_lu.cpp</code> (ReusableLU)	[IMPLEMENTED]
MKL PARDISO	default direct factorization; <code>mk1_rt</code> loaded at run time (no link-time dependency); threads via <code>PYTCAD_PARDISO_THREADS</code>	<code>core/src/solver/pardiso_lu.cpp</code> (PardisoLU)	[IMPLEMENTED]
scipy spsolve (SuperLU)	fallback / small systems; <code>PYTCAD_NATIVE_LINSOLVE=0</code> forces it	<code>linsolve.py</code>	[IMPLEMENTED]

Backend	What it is	Where	Status
GMRES / BiCGStab	Krylov with block-Jacobi or Schur-complement preconditioner (block size 3)	linsolve.py _build_block_jacobi_preconditioner:363, _build_schur_preconditioner:430	[IMPLEMENTED]
PETSc KSP/PC (and MUMPS)	optional compiled backend or petsc4py; MPI/GPU via PETSc not started (M31 P7)	core/src/solver/petsc_ksp.cpp; TCAD_WITH_PETSC=ON OFF	[PARTIALLY IMPLEMENTED] optional; no win-64 conda build, so Windows CI builds with PETSc OFF
GPU direct	cupy-based; soft import	linsolve.py (gpu_direct)	[PARTIALLY IMPLEMENTED] optional
MPI Schwarz domain decomposition	3D solve only	gui/services/mpi_schwarz_runner.py	[PARTIALLY IMPLEMENTED]

Policy: `DirectSession` (linsolve.py:1072) keeps symbolic analysis across Newton iterations but only above **MIN_UNKNOWNS=15000** (below that SuperLU is faster) and hands back to `spsolve` after **MAX_ANALYSES=2** pattern changes (nonlocal BTBT moves the pattern). Residuals above 10^{-6} raise `LinearSolveError`; non-finite systems raise too. **Auto selection** (`select_auto(dim, unstructured, coupled, dof)`:283) returns the benchmark-evidenced best method *and a reason string*; for any combination never measured it refuses to guess and returns direct (“Gate D-3”). The default is `NewtonOptions.linsolve='direct'`; ‘auto’ is opt-in. The evidence table is real Windows measurements from the repo: e.g. structured 3D Poisson equilibrium without PETSc, GMRES 2.81 s vs direct 187.3 s at 68,921 DOF; unstructured coupled 3D drift-diffusion, GMRES 1.28 s vs direct 7.38 s at 7,464 DOF (comments in linsolve.py). Krylov needs the physics-aware block preconditioners because the Jacobian is indefinite and badly scaled.

NUMERICAL IMPLICATION Fill-in and why 3D is hard

1D: block-tridiagonal, LU is $O(N)$. 2D: bandwidth $\propto N_x$ so LU is $O(N^{1.5})$ with nested-dissection ordering. 3D: $O(N^2)$ time, $O(N^{4/3})$ memory. Hence the C++ session reuse (symbolic once, numeric every iteration), the PARDISO default, and the Krylov/Schwarz options for large 3D.

Physics → equation → discretization → solver → code: Newton and linear algebra

Physics	Equation	Discretization	Solver	Source (file · class/function)
Nonlinear system	$F(u)=0, u=(\psi, n, p)$	—	Newton, analytic J	device1d.cpp newton :1542; device2d.py solve_bias :1923; device3d.py :1981
Contact constraints	Dirichlet DOFs	symmetric elimination	—	eliminate_dirichlet ; dirichlet.py
Linear solve	$J\delta = -F$	COO → CSR	LU (PARDISO/Eigen/SuperLU) or Krylov	core/src/solver/*; linsolve.py solve_linear :725, DirectSession :1072, select_auto :283
Options	tol_update, max_dpsi, max_iter, linsolve	—	—	device.py NewtonOptions :539

8. Beyond a single bias point

8.1 Bias continuation and arc-length continuation

[IMPLEMENTED] `adaptive_bias_sweep(device, v_end, step0)` (continuation.py:61) ramps a bias with step halving on failure — crucially *restoring the last converged state* before the retry (`solve_bias` mutates state even when it fails); it raises rather than stall. `arc_length_sweep` (:267) treats V as an unknown too, with a pseudo-arclength predictor and a **bordered corrector** (`_bordered_corrector`:124, a staged variant at :219 for generation-strength stages), and raises `ArcLengthStalled` when the corrector cannot proceed. Tests: `tests/test_m22_continuation.py`.

8.2 Transient simulation

Restoring $\partial n/\partial t$ in (3.3) and discretizing in time with a θ -scheme ($\theta=1$ is backward Euler, the only gated choice):

$$\theta F_n(\text{new}) + (1 - \theta)F_n(\text{old}) - dV \frac{n - n_0}{\Delta t} = 0, \quad \text{Poisson unchanged (algebraic)} \quad (8.1)$$

[IMPLEMENTED] `solve_transient` (transient.py:228, with `StepWaveform`, `RampWaveform`, `PulseWaveform`); 2D/3D in `transient2d.py` / `transient3d.py`. The storage term is added *post hoc* to the steady-state (F, J): the device class is untouched. Time scale $t_0 = N_s/R_0 = L_D^2/D_0$. Gates: FD-Jacobian, steady-state consistency, **charge conservation**, **dielectric relaxation time** ($\tau = \epsilon/\sigma$), and diode turn-off storage delay (`tests/test_m17_transient.py`). $\theta < 1$ is implemented but not gated.

8.3 Small-signal AC analysis

Linearizing around the DC operating point u_0 with a harmonic perturbation turns the time derivative into $j\omega$:

$$[J_0 + j\omega t_0 C_{\text{mat}}]\delta u = b, \quad Y(\omega) = \frac{\delta I}{\delta V} \quad (8.2)$$

[IMPLEMENTED] `ac_sweep`, `y_parameters`, `cutoff_frequency` (ac.py:130, 301, 393; 2D/3D in `ac2d.py`, `ac3d.py`). C_{mat} is exactly the transient storage matrix (gated numerically identical). The terminal-current sensitivity uses central finite differences over the six DOFs of the contact edge — correct for whatever model is on, at the price of not being an analytic derivative. Gates: low-frequency limit equals quasi-static DC derivative; junction capacitance matches the depletion formula $C = \epsilon A/W$; diffusion capacitance rolls off with frequency. Ohmic-port sensitivity: `pytcad/AC-SENSITIVITY-PLAN.md` (done, one open item).

8.4 Self-heating (electro-thermal)

Joule heating $H = J \cdot E$ raises the lattice temperature, which lowers mobility ($\mu \propto T^{-2.33}$) — a negative feedback that bends high-power I-V curves. Heat conduction: $-\nabla \cdot (\kappa(T) \nabla T) = H$ with $\kappa_{\text{th}}(T) = 1.48 (T/300)^{-1.33}$ W/(cm K) for Si (`Semiconductor.kappa_th`). Implementation choice, explicitly documented: **outer Gummel loop** between the isothermal electrical solve and the thermal solve, not a monolithic (ψ, n, p, T) Newton (`Device1D` builds its scaling once from a scalar T). **[IMPLEMENTED]**: `solve_lattice_temperature` (thermal.py:147), `joule_heating_density` (:177), `solve_electrothermal` (:220), boundary conditions `ThermalBC.isothermal/resistance/adiabatic`; 2D/3D in `thermal2d.py`, `thermal3d.py` with C++ thermal-grid assembly (`core/src/thermal/grid.cpp`).

Physics → equation → discretization → solver → code: dynamics and continuation

Physics	Equation	Discretization	Solver	Source (file · class/function)
Bias ramp	$V(s)$	adaptive step + restore	Newton per step	continuation.py adaptive_bias_sweep :61
Fold / breakdown	(u, V) with arc-length constraint	bordered system	Newton on augmented system	arc_length_sweep :267, _bordered_corrector :124
Transient	(3.3) with $\partial/\partial t$	backward Euler / θ	Newton per time step	transient.py solve_transient :228, _step_residual_jacobian :153
Small-signal AC	$(J_0 + j\omega C)\delta u = b$	complex linear solve	sparse LU	ac.py ac_sweep :130, y_parameters :301
Self-heating	$-\nabla \cdot (\kappa \nabla T) = J \cdot E$	1D/2D/3D FV heat	Gummel outer loop	thermal.py solve_electrothermal :220

9. Solver architecture: 1D, 2D and 3D

All three devices share one mathematical core — scaled (ψ , n , p) system, SG edge currents, Newton with symmetric Dirichlet elimination — but differ in how much runs in C++ and how the mesh is represented.

	Device1D	Device2D	Device3D
Class / file	Device1D (device.py:682) wraps the compiled pytcad._core.Device1D	Device2D (device2d.py:138)	Device3D (device3d.py:414)
Mesh	1D array x (graded)	Mesh2D tensor product, or gmsh triangles (unstructured=True)	Mesh3D box, or gmsh/level-set tetrahedra
Residual + Jacobian	C++: Device1D::residual_jacobian (device1d.cpp:950)	Python/numpy: _residual_jacobian (device2d.py:1402) with C++ kernels for thermal, DG, nonlocal paths	Python with C++ kernels: _residual_jacobian (device3d.py:1485); Poisson stencil in core/src/device3d/poisson_eq.cpp
Newton driver	C++ Device1D::newton:1542 / solve_bias:1708	Python solve_bias:1923	Python solve_bias:1981
Linear solve	dense-block tridiagonal LU in core (lu_solve:237)	linsolve.solve_linear, DirectSession above 15k unknowns	same; plus MPI Schwarz/GPU paths in gui/services
Contacts	2 ends	DirichletBC, GateBC, SchottkyBC	DirichletBC, PinnedBC, GateBC, SchottkyBC
Equilibrium	solve_equilibrium (Boltzmann/FD/DG)	:1308 (+ DG coupled)	:1311 (+ DG coupled; psi_guess)
Current	current_density	terminal_current(name):2194	terminal_current(name):2274

Dispatch hazards (from CLAUDE.md): shared dispatch code is dangerous — an unguarded 3D-only expression once broke every 1D/2D job; dimension-specific work is gated behind its own if and the whole suite is rerun after touching it. Model flags a dimension cannot honour **raise** rather than silently no-op (Models.__post_init__, Device2D/3D constructors).

9.1 A single bias solve, end to end (1D)

Step	What happens	Where
1	Device1D(x, doping, models=...): scaling, n_{ie} , μ , τ , edge diffusivities evaluated in C++	device.py:693; inputs.cpp (tcad::device1d inputs)
2	solve_equilibrium() — Poisson-only Newton; carriers slaved to ψ	device1d.cpp:557/572
3	solve_bias([V ₀ , V ₁]): contact values → ψ_0, n_0, p_0 ; warm start from the previous state; freeze TAT probabilities; (stiff models) strength ladder	device1d.cpp:1708
4	newton(): lagged mobility → assemble → Dirichlet elimination → lu_solve → damped update → update-norm test	device1d.cpp:1542
5	current_density, band_diagram, psi_V/n_cm3/p_cm3/E_field read the converged scaled state back to physical units	device.py:1443-1495

9.2 What lives outside the numerical core

workbench/: domain objects (core/device.py, region.py, materials.py, catalog.py, templates.py), adapters, solvers/base.py (the **SolverBackend** interface, only the pytcad backend since devsim was removed), analysis helpers that duplicate published-value physics (physics/impact_ionization.py, physics/tunneling.py), splits/batch/calibration/study manifests/compact-model extraction (M30, M38). **gui/services/**: DeviceSpec (the wire format), ResultStore, runners (subprocess per run → npz schema v3), examples, project store. **backend_service/server.py**: JSON-RPC 2.0 over stdio. **desktop/**: C++20/Qt 6 Widgets/VTK app (desktop/README.md, NATIVE-DESKTOP-PLAN.md; M53 in progress). None of these implement physics; they package it.

10. Advanced physics models

10.1 Impact ionization and avalanche breakdown

At high field a carrier gains enough energy between collisions to knock a new electron-hole pair across the gap. The pair generation rate grows with the carrier flux and with an *ionization coefficient* $\alpha(E)$ (pairs per carrier per cm):

$$G_{II} = \frac{1}{q}(\alpha_n |J_n| + \alpha_p |J_p|), \quad \alpha(E) = A e^{-B/E} \quad (10.1)$$

[IMPLEMENTED] model; **[PARTIALLY IMPLEMENTED]** breakdown-voltage extraction.

`pytcad/ionization.py` holds the van Overstraeten-de Man table (A in cm^{-1} , B in V/cm): electrons (7.03×10^5 , 1.231×10^6) over $[1.75 \times 10^5, 6.0 \times 10^5] \text{ V/cm}$, a single branch; holes (1.582×10^6 , 2.036×10^6) below $E = 4.0 \times 10^5 \text{ V/cm}$ and (6.71×10^5 , 1.693×10^6) above. The hole switch point was a **real bug** fixed 2026-08-28 (the code shared electrons' 5×10^5 and misclassified a field window); the module docstring keeps the story — a good example of why published constants are pinned by tests

(`test_alpha_coefficients_match_published_table`). Local form: `Models(impact=True)`. Nonlocal (M34-S2): α is evaluated at a per-carrier *effective* field from the relaxation equation $\lambda dE_{\text{eff}}/ds + E_{\text{eff}} = |E|$ along the drift direction, $\lambda = 650 \text{ \AA}$ (Slotboom et al., IEDM 1991; only the abstract was accessible — the module says so) — `Models(impact_nonlocal=True)`, `pytcad/ii_nonlocal.py`, `ii_nonlocal_grid.py`, C++ `core/src/nonlocal/*`.

Coupling: G_{II} is *live* in the Jacobian (the R1b fix), not lagged. In the C++ assembly the source at node j is $K_{\text{gen}}(\alpha_n(E_j) \cdot S_{n,j} + \alpha_p(E_j) \cdot S_{p,j})$, S = sum of adjacent-edge $|J|/2$, $K_{\text{gen}} = 1/(2qR_0)$ (`device1d.cpp` ~line 1216). $|J|$ is regularized as $\sqrt{(J^2 + \epsilon^2)}$ with $\epsilon = 10^{-6} \cdot \max |J|$ (`kIiJEpsRel`) so the derivative of $|J|$ is smooth through zero. 2D/3D: `pytcad/ii_grid.py` (one kernel for both).

Physical picture. Impact ionization is a positive-feedback process: more carriers $\rightarrow$ more current $\rightarrow$ more generation. The multiplication factor $M = 1/(1 - \int \alpha dx)$ diverges when the ionization integral reaches 1: *avalanche breakdown*. Analytic reference in the repo: `ionization_integral(V, N)` and `breakdown_voltage_one_sided(N)` (`workbench/physics/impact_ionization.py`).

CAUTION Open gate G-D (read this)

The solver-side breakdown test `tests/test_m15_ionization.py::test_g_d_breakdown_within_ten_percent` is the one known failing gate. For $N = 2 \times 10^{16} \text{ cm}^{-3}$ the analytic breakdown is 33.7 V while the arc-length corrector stalls near 41.9 V on the current build (ARCHITECTURE.md §6). The traced current shows no multiplication before the stall — the “fold” the continuation stops at is decided by round-off in near-cancelling junction edge currents (§7.5 caution), not by physical breakdown. What is needed is a physical breakdown criterion (e.g. multiplication M) and a round-off-safe regularizer. This guide does not claim it works; the *model* and the *continuation machinery* are implemented, the *automatic breakdown detection* is not yet right.

10.2 Band-to-band tunneling (BTBT)

In a strongly reverse-biased or heavily doped junction, valence-band electrons tunnel directly into the conduction band across the gap (Zener tunneling, GIDL in MOSFETs). The local Kane/Hurkx form uses the local field F :

$$G_{BTBT}(F) = A F^2 e^{-B/F}, \quad A = 3.5 \times 10^{21} \text{ cm}^{-3}\text{s}^{-1}, \quad B = 1.03 \times 10^8 \text{ V/cm} \quad (10.2)$$

[IMPLEMENTED] Local: `Models(btbt=True)`; `btbt.btbt_generation / dbtbt_dF` (derivative $G(2/F + B/F^2)$); 2D/3D grid version `btbt_grid.py`. **Nonlocal path BTBT [IMPLEMENTED]** (1D homojunction first; 2D/3D in M34-S3): a WKB integral along the carrier's tunnel path with the Kane two-band complex wave-vector $\kappa(E)$ — first-principles, not calibrated to the empirical A, B — `Models(btbt_nonlocal=True)`, `btbt.nonlocal_btbt_window:336`, `segment_integrals`, `kane_kappa`, `nonlocal_path.py`, C++ `core/src/nonlocal/{paths,segments,evaluate}.cpp`. It reduces exactly to its own published uniform-field closed form (gated: `test_nonlocal_btbt_reduces_to_published_kane_formula_in_uniform_field`). Tunnel windows are located once per solve and frozen; after convergence they are re-located and the solve repeated (≤ 4 refreshes, outcome reported). *Known limit (module docstring)*: the local model underestimates leakage at large reverse bias because one local field stands in for the whole path; it is gated on its failure mode (current does not plateau).

10.3 Tunneling through oxides and barriers

[IMPLEMENTED] (analysis layer) `workbench/physics/tunneling.py`: Fowler-Nordheim constant, WKB decay length, direct-tunneling limits (tests in `test_model_benchmarks.py`: `test_fn_constant_matches_physical_definition`, `test_wkb_decay_length_in_published_band`). It is gate-level analysis for oxides, not a transport boundary condition inside Device2D. Gate-leakage currents through the oxide are therefore **[NOT IMPLEMENTED]** as a solver-coupled effect.

10.4 Quantum confinement: density gradient and Schrödinger-Poisson

In thin inversion layers carriers are confined within a few nm; classical theory puts the density peak at the oxide interface, whereas quantum mechanics pushes it away (the *dark space*) and raises threshold voltage. The density-gradient (Ancona-Stafford) model adds a quantum potential (Bohm potential) to the effective electrostatic potential seen by the carriers:

$$n = n_{cl} e^{-\Lambda_n/V_T}, \quad \Lambda_n = -\frac{\gamma \hbar^2}{2m^*q} \frac{\nabla^2 \sqrt{n}}{\sqrt{n}} \quad (10.3)$$

[PARTIALLY IMPLEMENTED] Equilibrium only: `Models(dg=True)` works in 1D/2D/3D for equilibrium and MOSCapacitor C-V (`MOSCapacitor(dg=True)`); `solve_bias` raises — DG transport is out of scope. The correct formulation is a *coupled* Newton on $(\psi, \Lambda_n, \Lambda_p)$: an earlier lagged outer iteration converged cleanly to *wrong* physics (recorded in the module and in `dg.py`; M20 plan §6). $|\Lambda|$ is clamped at $20 V_T$ — measured to be **load-bearing** at the surface (raw curvature reaches $\sim 81 V_T$). DG with Fermi-Dirac, and DG with the affinity gauge, are refused (unvalidated compositions). Functions: `dg.quantum_potential`, grid versions `dg_grid.py`, C++ `core/src/dg/grid.cpp`. A **self-consistent 1D Schrödinger-Poisson** solver (`dg.schrodinger_poisson`:135, `schrodinger_poisson_mos`:315) and a closed-form Airy triangular-well reference (`airy_triangular_well`) serve as published-value gates for DG.

10.5 Hot carriers: energy balance and the hydrodynamic closure

Two distinct things exist and should not be confused. (a) `pytcad/hydrodynamic.py` (M29) is a *post-processing* local closure: carrier temperature from the local steady energy balance (no energy-flux term), an energy relaxation length $l_w = v_{sat} \tau_w$, and carrier-temperature-based impact ionization mapped back to an effective field; the module's own docstring states it cannot reproduce overshoot spatial shape. (b)

`Models(energy_balance=True)` (M44; 1D/2D/3D): a live **fourth unknown** $\theta = T_n/T$ appended after the (ψ, n, p) block (DOF indices $3N \dots 4N-1$, so the base block's math is unchanged and default-off is bit-identical), with lagged Joule-heating source $J_n \cdot E_n$ and a T_n -dependent field for Canali mobility. **[PARTIALLY IMPLEMENTED]**: electron-only; refused together with impact/BTBT; coefficients lagged one iterate.

Physics → equation → discretization → solver → code: advanced physics

Physics	Equation	Discretization	Solver	Source (file · class/function)
Impact ionization	$G = (\alpha_n J_n + \alpha_p J_p)/q$	nodal, live Jacobian, $\sqrt{(J^2 + \epsilon^2)}$	Newton + strength ladder + line search + arc-length	<code>ionization.py</code> ; <code>device1d.cpp</code> (~1216); <code>physics/generation.hpp</code> ii_alpha ; <code>ii_grid.py</code>
Nonlocal II	$\lambda dE_{eff}/ds + E_{eff} = E $	effective-field per carrier (dense Jacobian block)	Newton	<code>ii_nonlocal.py</code> ; <code>device1d.cpp</code> effective_field :349
Local BTBT	$G = AF^2 e^{-B/F}$	nodal from edge field	Newton	<code>btbt.py</code> btbt_generation ; <code>btbt_grid.py</code>
Nonlocal BTBT	WKB path integral	path windows, frozen per solve	Newton + refresh	<code>btbt.py</code> nonlocal_btbt_window ; <code>nonlocal_path.py</code> ; <code>core/src/nonlocal/*</code>
Density gradient	(10.3)	coupled $(\psi, \Lambda_n, \Lambda_p)$	coupled Newton (equilibrium)	<code>dg.py</code> ; <code>dg_grid.py</code> ; <code>core/src/dg/grid.cpp</code> ; <code>device1d.cpp</code> dg_newton :844
Energy balance	$\theta = T_n/T$, 4th DOF	lagged coefficients	Newton	<code>Models.energy_balance</code> ; <code>device1d.cpp</code> (eb), <code>hydro_grid.py</code>

11. Process simulation: where the doping comes from

Device behaviour is dominated by the doping profile; process TCAD predicts it from fabrication steps and hands it to the device solver (the same relationship Sentaurus Process has to Sentaurus Device). The repository implements it at **teaching/engineering** fidelity, with honest limits.

Step	Physics	Model in repo	Source	Status
Ion implantation	stopping of ions in the crystal → depth profile	Gaussian/Pearson-IV profile with approximate LSS moments for amorphous Si (5-10% accurate; no channeling)	process.py implant, implant_moments, implant_pearson4_skewed	[IMPLEMENTED]
Monte-Carlo implant	binary-collision approximation	BCA ion tracking, channeling in crystal; range same order as SRIM	mc_implant.py; test_m25_mc_implant.py	[PARTIALLY IMPLEMENTED] 1D slice
Diffusion	Fick's law $\partial C/\partial t = \partial/\partial x(D \partial C/\partial x)$, $D(T)$ Arrhenius	erfc predep, Gaussian drive-in, numeric solver (C++ kernel)	process.py diffuse_*; core/src/process/diffuse.cpp	[IMPLEMENTED]
TED / defects	transient enhanced diffusion by interstitials	lumped Fair-type extrinsic enhancement + segregation; not coupled point-defect PDEs	ted.py; test_m24_ted.py	[PARTIALLY IMPLEMENTED]
Oxidation	Deal-Grove $x^2 + Ax = B(t + \tau)$; Si consumed 0.44 x	1D analytic; 2D/3D level-set moving boundary; thin dry oxide under-predicted	process.py oxide_thickness; oxidize_levelset*.py	[PARTIALLY IMPLEMENTED]
Geometry evolution	moving interfaces	level-set deposit / etch / oxidation / silicidation / CMP / epitaxy (2D & 3D)	levelset2d.py, levelset3d.py, silicide_levelset*.py, process2d.py	[IMPLEMENTED]
Stress / strain	mechanical stress effects on mobility	—	(M36 not started)	[NOT IMPLEMENTED]

PHYSICAL INTUITION Why the device simulator needs N_{total}

A diffused profile has donors and acceptors overlapping. Net doping ($N_D - N_A$) sets the electrostatics, but mobility and lifetime depend on *total* ionized impurities. README example: Device1D(x, C-1e16, Ntotal=C+1e16).

12. Verification: how you know the physics is right

The repository holds ~1207 test functions in `tests/` (132 files) and ~685 in `gui/tests/` (static counts of `def test_`). Policy (CLAUDE.md): new physics lands in `tests/test_model_benchmarks.py` *first* with published constants; numerical paths are gated by **FD-Jacobian**, **reduction** and **bit-identity** tests; the pytest suite must end **N passed, zero warnings**; tests are *never weakened*.

Gate type	Question it answers	Example
FD-Jacobian	Is the analytic J the exact derivative of F?	<code>test_g0_jacobian_matches_finite_differences</code> ; <code>test_g_gen_jacobian_matches_finite_differences</code>
Analytic / published value	Does the model reproduce a known number?	built-in potential (G1); alpha table; Kane A,B; Richardson A_0 ; Deal-Grove
Physical identity	Does it conserve what physics conserves?	global charge neutrality, Gauss surface balance (MOS-C); charge conservation (transient)
Limit / reduction	Does a richer model collapse to the simple one?	3D→2D→1D; nonlocal BTBT → uniform-field Kane; Boltzmann ↔ FD at low doping; TAT with $P=0$ → SRH
Bit-identity (model off)	Does turning a model off change nothing?	Models flags default OFF ⇒ goldens unchanged
Convergence	Does refinement converge?	mesh refinement (G4), V-step and Newton-tolerance refinement (MOS-C)
Parity	Does the C++ path match the Python reference?	<code>tests/test_accel_parity.py</code> , <code>test_device3d_poisson_eq_accel_parity.py</code>

Goldens (*.npz, gitignored) pin one machine's floating-point order; they must be regenerated on the target machine and never copied. Before a core edit the md5 of every golden it could touch is recorded, the edit made, the old behaviour shimmed back to prove the old md5s return, and any moved golden explained. This discipline exists because round-off is a physical-correctness hazard in drift-diffusion (§7.5).

CI: `.github/workflows/ci.yml` runs a *tests* job and a *wheel* job on windows-latest; `_core` is built with conda-forge MinGW g++ in a separate `tcad-cpp` env; PETSc OFF. The only known failure is G-D (§10.1).

13. Physics implemented in the current TCAD

Status judged from source, tests and ARCHITECTURE.md (capability matrix, §3 there). “1/2/3” = dimensions where the feature runs and is gated.

Feature	Status	Dim.	Where (principal source)	Notes / limits
Poisson electrostatics	[IMPLEMENTED]	1/2/3	device1d.cpp; device2d.py:844; device3d.py:952; poisson_eq.cpp	box FV, scaled by L_D
Drift-diffusion + continuity (Scharfetter-Gummel)	[IMPLEMENTED]	1/2/3	device1d.cpp residual_jacobian; device2d/3d_residual_jacobian	fully coupled Newton
Structured meshes	[IMPLEMENTED]	1/2/3	mesh.py, mesh2d.py, mesh3d.py	tensor product
Unstructured meshes (gmsh)	[IMPLEMENTED]	2/3	unstructured_*.py; core/src/unstructured3d	3D homojunction + hetero (M47-S3)
Adaptive mesh refinement	[IMPLEMENTED]	1/2/3	adapt*.py; core/src/process/indicators.cpp	
Boltzmann statistics	[IMPLEMENTED]	1/2/3	default	warn $>10^{19} \text{ cm}^{-3}$
Fermi-Dirac statistics	[IMPLEMENTED]	1/2/3	fermi.py; physics/fermi.hpp	parabolic-band $F_{1/2}$; not with affinity gauge/DG
Incomplete ionization	[IMPLEMENTED]	1/2/3	device.py ionized_doping	hydrogenic; invalid $>4 \times 10^{18}$
Bandgap narrowing (Slotboom)	[IMPLEMENTED]	1/2/3	materials.py	default on
SRH + Auger recombination	[IMPLEMENTED]	1/2/3	materials.py; physics/kernels.hpp	default on; Scharfetter lifetimes
Radiative recombination	[NOT IMPLEMENTED]	—	—	absent
Trap-assisted tunneling (Hurkx/WKB)	[IMPLEMENTED]	1/2/3	device1d.cpp tat_probabilities	probabilities frozen per solve
Surface recombination velocity (Robin)	[PARTIALLY IMPLEMENTED]	1/2	device.py/device2d.py	Device3D refuses
Caughey-Thomas doping mobility	[IMPLEMENTED]	1/2/3	materials.py	use $N_A + N_D$
Canali field mobility	[PARTIALLY IMPLEMENTED]	1	materials.py; device1d.cpp	lagged; 2D/3D refuse
Lombardi CVT surface mobility	[PARTIALLY IMPLEMENTED]	2	materials.py mobility_cvt	absolute calibration open (M14 G-A)
Heterojunctions (Anderson, nie/affinity gauges)	[IMPLEMENTED]	1/2/3	device1d.cpp band_shift_; workbench/core/materials.py	materials: Si, Ge, GaAs, InGaAs, AlGaAs
Thermionic interface flux	[IMPLEMENTED]	1	device1d.cpp (te_*)	affinity gauge only; refused with impact
Ohmic contacts	[IMPLEMENTED]	1/2/3	contact_value; DirichletBC	ideal
Gate (Robin) boundary / MOS	[IMPLEMENTED]	1/2/3	GateBC; moscap.py	oxide not meshed
Schottky contacts	[PARTIALLY IMPLEMENTED]	1/2/3	schottky.py; SchottkyContact/BC	3D: Dirichlet only (Robin missing)
MOS capacitor C-V (quasi-static), D_{it} , Q_f	[IMPLEMENTED]	1	moscap.py	no HF solve
Impact ionization, local (vOdM)	[IMPLEMENTED]	1/2/3	ionization.py; ii_grid.py	
Impact ionization, nonlocal effective field	[IMPLEMENTED]	1/2/3	ii_nonlocal*.py	$\lambda_p = \lambda_n$ simplification
Breakdown-voltage detection (arc-length)	[PARTIALLY IMPLEMENTED]	1	continuation.py; test_m15_ionization.py	G-D open
BTBT local (Kane/Hurkx)	[IMPLEMENTED]	1/2/3	btbt.py; btbt_grid.py	underestimates high reverse bias
BTBT nonlocal (WKB path)	[IMPLEMENTED]	1/2/3	btbt.py; nonlocal_path.py; core/src/nonlocal	homojunction only
Density gradient (Ancona)	[PARTIALLY IMPLEMENTED]	1/2/3	dg.py; dg_grid.py	equilibrium only
Schrödinger-Poisson (analysis)	[IMPLEMENTED]	1	dg.py	published-value gate for DG

Feature	Status	Dim.	Where (principal source)	Notes / limits
Energy balance (electron T_n)	[PARTIALLY IMPLEMENTED]	1/2/3	Models.energy_balance	electron-only; not with impact/BTBT
Self-heating	[IMPLEMENTED]	1/2/3	thermal*.py; core/src/thermal	outer Gummel loop (disclosed)
Transient	[IMPLEMENTED]	1/2/3	transient*.py	backward Euler gated
Small-signal AC / Y-parameters	[IMPLEMENTED]	1/2/3	ac*.py	FD terminal sensitivity
Continuation (adaptive, arc-length)	[IMPLEMENTED]	1/2/3	continuation.py	
Mixed-mode circuit (MNA)	[IMPLEMENTED]	1/2/3	circuit.py	FD conductance companion model
Implant, diffusion, Deal-Grove	[IMPLEMENTED]	1	process.py	LSS moments, no channeling
TED, Monte-Carlo implant	[PARTIALLY IMPLEMENTED]	1	ted.py, mc_implant.py	lumped / 1D
Level-set process (deposit/etch/oxidation/silicide)	[IMPLEMENTED]	2/3	levelset*.py	
Linear solvers: direct (PARDISO/Eigen/SuperLU), GMRES/BiCGStab	[IMPLEMENTED]	—	linsolve.py; core/src/solver	
PETSc / MUMPS / GPU / MPI	[PARTIALLY IMPLEMENTED]	—	petsc_ksp.cpp; gui/services	optional; P7 not started
Compact-model extraction, calibration, splits/batch	[IMPLEMENTED]	—	workbench/compact.py, calibration.py, splits.py, batch.py	M30, M38

14. Missing and future physics

Everything below is **[NOT IMPLEMENTED]** in the current repo (verified by search or ARCHITECTURE.md), grouped by what you would need to add it.

Missing physics	Why it matters	What implementing it would involve
Radiative recombination, photogeneration, optics (M40)	LEDs, lasers, solar cells, photodetectors	a $G_{\text{opt}}(x)$ source (Beer-Lambert / transfer matrix) and $B \cdot (np - n_{\text{ie}}^2)$ in R; plugs into existing continuity rows
Stress/strain (M36)	strained-Si mobility, band-structure shifts	elasticity solve + deformation-potential shifts to E_c , E_v , m^* , μ
Reliability: BTI, HCI, TDD (M37)	aging, lifetime prediction	interface/oxide trap kinetics coupled in time to the existing D_{it} / Q_f hooks
Quantum transport (NEGF, M39)	ballistic / few-nm channels, tunnel FETs	non-equilibrium Green's functions, Schrödinger equation with open boundaries
DG transport (not just equilibrium)	quantum confinement under bias	carry Λ through the coupled Newton in solve_bias (currently refused)
Hole energy balance; full hydrodynamic with flux term	velocity overshoot, hot-hole effects	extend the 4th-DOF block to T_p and add energy-flux divergence
Monte-Carlo Boltzmann transport	reference for hot-carrier physics	out of scope per ARCHITECTURE.md §7
Coupled point-defect diffusion, kinetic MC	accurate TED, clustering	reaction-diffusion system for I and V, replacing the lumped model
Gate tunneling current as a solver boundary term	gate leakage in thin oxides	couple tunneling.py's WKB/FN currents into GateBC as a carrier flux
Canali field mobility in 2D/3D	velocity saturation in MOSFET channels	port the lagged Canali update to the 2D/3D drivers (currently refused)
Schottky Robin contact in 3D; surface recombination in 3D	completeness across dimensions	port the Device2D Robin machinery
High-frequency C-V	accurate MOS C-V in inversion	explicit minority-carrier freeze-out (only quasi-static + gated QS/HF comparison now)
Physical breakdown criterion (G-D)	robust BV extraction	detect $M \rightarrow \infty$ or $f\alpha=1$ and use a round-off-safe regularizer (open item)
Absolute surface-mobility calibration (G-A)	accurate inversion-layer μ	needs the paywalled Lombardi 1988 constants
Adjoint sensitivities, gradient-based calibration, UQ, ML surrogates	optimisation and inverse design	proposed next per ARCHITECTURE.md; symmetric Dirichlet elimination was introduced to keep J^T usable for adjoints
Finite-element discretization	complex geometries with curved boundaries	not planned; FV on tensor-product / dual meshes is the chosen method
Electromagnetic solvers, radiation effects, ferroelectric/phase-change materials, foundry-grade PDK compact models	specialist domains	out of scope per ARCHITECTURE.md §7
PETSc-based MPI/GPU scaling (M31 P7)	very large 3D	not started

15. Physics learning dependency map

Learning dependency map: an arrow means “understand this first”

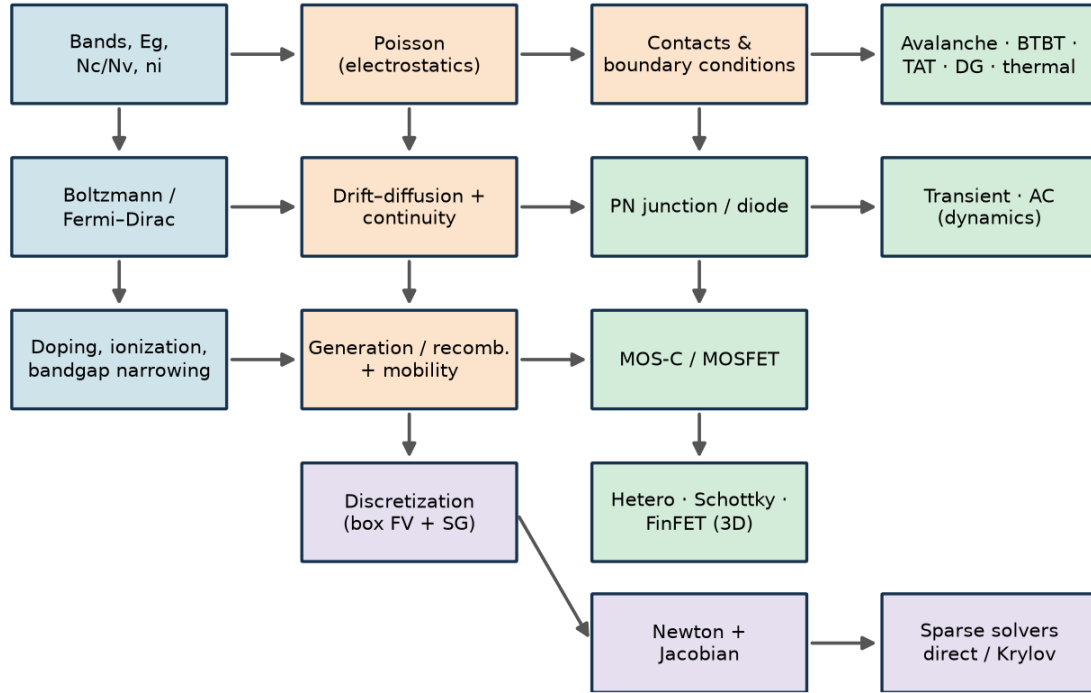


Figure 11 — Prerequisites. Blue: material fundamentals. Orange: the three governing ideas (electrostatics, transport, G/R+mobility, contacts). Green: devices and extensions. Purple: numerics.

Suggested study path (with repository exercises)

#	Learn	Then do in the repo	Checkpoint
1	Bands, E_g , n_i , mass action (§1)	Compute <code>SILICON.ni(300)</code> , vary T; read <code>materials.py</code>	$n_i \approx 1.07 \times 10^{10}$ at 300 K
2	Doping, neutrality, contacts (§1.3, §5)	<code>Device1D.solve_equilibrium()</code> ; print n,p at the ends	$n_0 = \frac{1}{2}(C + \sqrt{C^2 + 4n_{ie}^2})$
3	Poisson, depletion, V_{bi} (§2)	Compare solver V_{bi} and E_{max} to eq. (2.2)	7-digit V_{bi} match
4	Drift-diffusion, quasi-Fermi levels (§3)	<code>band_diagram()</code> at 0 V and 0.5 V (Fig. 3)	E_{Fn}/E_{Fp} split by qV
5	G/R, mobility (§4)	<code>Toggle Models(srh, auger, bgn)</code> ; plot I-V	ideality 1→2 at low bias
6	Discretization, SG (§7.3–7.4)	Read <code>kernels.bernoulli</code> ; coarsen the mesh and watch SG stay stable	no negative densities
7	Newton, Jacobian (§7.5–7.6)	<code>NewtonOptions(verbose=True)</code> trace; run the FD-Jacobian test	quadratic convergence near solution
8	MOS physics (§6.2)	<code>MOSCapacitor.cv_sweep</code> ; check C_{min} , V_{fb}	Fig. 6 shape
9	2D/3D devices, linear algebra (§7.7, §9)	<code>examples/04_mosfet_idvg.py</code> , <code>05_3d_reduces_to_2d.py</code>	3D reproduces 2D
10	Advanced: II, BTBT, DG, thermal (§10, §8.4)	Read <code>ionization.py</code> , <code>btbt.py</code> ; run <code>examples/debug_m15_*.py</code>	understand G-D

16. Semiconductor / TCAD equation cheat sheet

Topic	Equation	Code symbol / location
Thermal voltage	$V_T = \frac{kT}{q} = 25.852 \text{ mV (300 K)}$	constants.thermal_voltage
Bandgap (Varshni)	$E_g(T) = E_g(0) - \frac{\alpha T^2}{T + \beta}$	Semiconductor.Eg
Intrinsic density	$n_i = \sqrt{N_c N_v} e^{-E_g/2kT}$	Semiconductor.ni
Boltzmann carriers	$n = n_i e^{(E_F - E_i)/kT}, p = n_i e^{(E_i - E_F)/kT}, np = n_i^2$	slaved densities (equilibrium)
Fermi-Dirac	$n = N_c F_{1/2}(\eta_n)$	fermi.f_half
Neutral contact density	$n_0 = \frac{1}{2} \left(C + \sqrt{C^2 + 4n_{ie}^2} \right)$	contact_value
Bandgap narrowing	$n_{ie}^2 = n_i^2 e^{\Delta E_g/kT}$	bandgap_narrowing_slotboom
Poisson	$\nabla \cdot (\epsilon \nabla \psi) = -q(p - n + C)$	Poisson row F[3i]
Debye length	$L_D = \sqrt{\epsilon V_T / (qN)}$	inputs.cpp LD
Built-in potential	$V_{bi} = V_T \ln(N_A N_D / n_i^2)$	—
Depletion width	$W = \sqrt{\frac{2\epsilon V_{bi}}{q} \left(\frac{1}{N_A} + \frac{1}{N_D} \right)}$	—
Drift-diffusion	$J_n = q\mu_n(nE + V_T \nabla n), J_p = q\mu_p(pE - V_T \nabla p)$	Jn[k], Jp[k]
Quasi-Fermi form	$J_n = -q\mu_n n \nabla \phi_n$	—
Continuity	$\nabla \cdot J_n = q(R - G), \nabla \cdot J_p = -q(R - G)$	F[3i+1], F[3i+2]
SRH	$R = \frac{np - n_{ie}^2}{\tau_p(n + n_{ie}) + \tau_n(p + n_{ie})}$	recombination
Auger	$R = (C_n n + C_p p)(np - n_{ie}^2)$	Cn_auger, Cp_auger
Caughey-Thomas	$\mu = \mu_{min} + \frac{\mu_{max} - \mu_{min}}{1 + (N/N_{ref})^\alpha}$	mobility_caughey_thomas
Canali	$\mu(E) = \mu_0 [1 + (\mu_0 E / v_{sat})^\beta]^{-1/\beta}$	mobility_field
Scharfetter-Gummel	$J_n = \frac{\mu_n V_T}{h} [n_{k+1} B(\delta) - n_k B(-\delta)]$	Jn[k]
Bernoulli	$B(x) = \frac{x}{e^x - 1}$	bernoulli
Scaling	$\hat{\psi} = \psi / V_T, \hat{n} = n / N_s, \hat{x} = x / L_D$	inputs.cpp
Newton step	$J(u^k) \delta = -F(u^k), u^{k+1} = u^k + \delta$	newton()
Stop criterion	$\max(\delta \psi , \delta n/n , \delta p/p) < 10^{-8}$	NewtonOptions.tol_update
Transient (BE)	$F - dV \frac{n - n_0}{\Delta t} = 0$	transient_step_residual_jacobian
AC small signal	$(j\omega + j\omega t_0 C) \delta u = b$	ac.ac_sweep
Impact ionization	$G = \frac{\alpha_n J_n + \alpha_p J_p }{q}, \alpha = A e^{-B/E}$	ionization.py
Breakdown condition	$\int_0^W \alpha dx = 1$	ionization_integral
Kane BTBT	$G = A F^2 e^{-B/F}$	btbt_generation
Density gradient	$n = n_{cl} e^{-\Lambda/V_T}, \Lambda = -\frac{\gamma \hbar^2}{2m^* q} \frac{\nabla^2 \sqrt{n}}{\sqrt{n}}$	dg.quantum_potential
MOS C-V	$C_{ox} = \epsilon_{ox} / t_{ox}, V_g = V_{fb} + \psi_s + Q_s / C_{ox}$	MOSCapacitor
Heat equation	$-\nabla \cdot (\kappa \nabla T) = J \cdot E, \kappa = \kappa_{300} (T/300)^{-1.33}$	thermal.solve_lattice_temperature
Deal-Grove	$x^2 + Ax = B(t + \tau)$	process.oxide_thickness

17. Repository architecture map

Paths relative to pytcad/ (the project root inside the repository). Top level of the repo: README.md, ARCHITECTURE.md, CLAUDE.md, .github/workflows/ci.yml.

```
pytcad/
├─ pytcad/                numerical core (70 modules)
│   ├── device.py         DeviceID (wrapper of compiled solver), Models, NewtonOptions, SchottkyContact
│   ├── device2d.py       Device2D, DirichletBC, GateBC, SchottkyBC
│   ├── device3d.py       Device3D, + PinnedBC
│   ├── materials.py      Semiconductor, mobility, BGN, lifetimes, recombination
│   ├── constants.py · kernels.py (Bernoulli, FD) · fermi.py · dirichlet.py
│   ├── ionization.py · ii_grid.py · ii_nonlocal*.py    impact ionization
│   ├── btb.t.py · btb.t_grid.py · nonlocal_path.py     tunneling
│   ├── dg.py · dg_grid.py · hydrodynamic.py · hydro_grid.py
│   ├── moscap.py · mosfet.py · finfet3d.py · umos3d.py · sic_vmosfet.py
│   ├── mesh.py · mesh2d.py · mesh3d.py · gmsh_mesh*.py · adapt*.py · unstructured*.py
│   ├── linsolve.py · continuation.py · transient*.py · ac*.py · thermal*.py · circuit.py
│   ├── process.py · process2d.py · ted.py · mc_implant.py · levelset*.py · oxidize*.py
│   └─ _accel.py (loads pytcad_core) · errors.py · characterization.py
├─ core/                  C++ engine → pytcad_core (nanobind + Eigen; PARDISO runtime; PETSc optional)
│   ├── include/tcad/ + src/{deviceid, physics, solver, nonlocal, thermal, dg, process,
│   │                   device3d, unstructured3d, mesh, geom, runtime, uicore, base}
│   └─ bindings/ · CMakeLists.txt
├─ workbench/             domain layer: core, adapters, solvers, analysis, physics, batch, ...
├─ gui/services/          Qt-free logic: DeviceSpec, ResultStore, runners, run_config, project_store
├─ backend_service/       JSON-RPC 2.0 over stdio
├─ desktop/              C++20 / Qt 6 Widgets / VTK app
├─ tests/ (132 files)     core validation    gui/tests/ (84 files)    services, RPC, desktop
├─ benchmarks/           performance suite + BASELINE.md            examples/    runnable examples
```

Physics → file lookup

I want to read...	Open
Poisson + SG + continuity assembly (1D)	core/src/device1d/device1d.cpp — residual_jacobian (950)
Newton loop, line search, stage ladder	device1d.cpp — newton (1542), solve_bias (1708); header: core/include/tcad/device1d/device1d.hpp (kliStages...)
Scaling constants	core/src/device1d/inputs.cpp (lines 41–65)
Bernoulli, recombination kernels	core/include/tcad/physics/kernels.hpp; core/src/physics/kernels.cpp; pytcad/kernels.py
Fermi–Dirac, ionized doping	pytcad/fermi.py; core/include/tcad/physics/fermi.hpp
Impact-ionization α and generation	pytcad/ionization.py; core/include/tcad/physics/generation.hpp
2D / 3D assembly	pytcad/device2d.py (_residual_jacobian :1402); pytcad/device3d.py (:1485); core/src/device3d/
Direct / iterative solvers	pytcad/linsolve.py; core/src/solver/{direct_lu,pardiso_lu,petsc_ksp}.cpp
Continuation / transient / AC / thermal	pytcad/{continuation,transient,ac,thermal}.py
MOS capacitor	pytcad/moscap.py
Process simulation	pytcad/{process,process2d,tcd,mc_implant,levelset2d,levelset3d}.py; core/src/process/
Wire format / backend / desktop	gui/services/device_spec.py; backend_service/server.py; desktop/
Rules, status, open items	CLAUDE.md; ARCHITECTURE.md; pytcad/*-PLAN.md (4 plans remain in tree)

NOTE Where to go next

Start with examples/01_pn_diode.py, then read deviceId.cpp residual_jacobian side by side with §3 and §7.4-7.5 of this guide; run tests/test_deviceId_native_gates.py to see each physical identity checked. Removed milestone plans (the long physics write-ups) are recoverable from git history: `git log --all -- <path>` then `git show <commit>^:<path>` (after `git fetch --unshallow` in a shallow clone).